

INTERACTIONS ENTRE L'HOMME ET LA MACHINE AVEC UNE APPLICATION WEB

1. LE WEB

Le « World Wide Web », plus communément appelé « Web » a été développé au CERN (Conseil Européen pour la Recherche Nucléaire) par le Britannique Sir Timothy John Berners-Lee et le Belge Robert Cailliau au début des années 90. A cette époque les principaux centres de recherche mondiaux étaient déjà connectés les uns aux autres, mais pour faciliter les échanges d'information Tim Berners-Lee met au point le système hypertexte. Le système hypertexte permet, à partir d'un document, de consulter d'autres documents en cliquant sur des mots clés. Ces mots « cliquables » sont appelés hyperliens et sont souvent soulignés et en bleu. Ces hyperliens sont plutôt connus aujourd'hui sous le simple terme de « liens ».

Cette première page Web est toujours consultable à l'adresse suivante :

<http://info.cern.ch/hypertext/WWW/TheProject.html>

Tim Berners-Lee a développé le premier navigateur web (logiciel permettant de lire des pages contenant des hypertextes), il l'a appelé « WorldWideWeb ». Il faudra attendre 1993 et l'arrivée du navigateur web « NCSA Mosaic » pour que le web commence à devenir populaire en dehors du monde de la recherche.

Techniquement le web se base sur trois technologies : le protocole HTTP (HyperText Transfert Protocol), les URL (Uniform Resource Locator) et le langage de description HTML (HyperText Markup Language).

Beaucoup de personnes confondent « Web » et « Internet » mais les deux choses n'ont rien à voir. Le « Web » désigne un ensemble de données reliées entre elles par des liens hypertextes et accessible sur Internet. « Internet » est un « réseau de réseau » s'appuyant sur le protocole IP (Internet Protocol) et sur lequel on trouve les applications comme le Web, les emails avec le protocole SMTP (Simple Mail Transfert Protocol) et les transferts de fichiers avec le protocole FTP (File Transfert Protocol).

2. L'URL

Dans la barre d'adresse de votre navigateur web vous trouverez, quand vous visitez un site, des choses de ce type : « <http://www.lyc-turgot.ac-limoges.fr/spip.php?article2825> ».

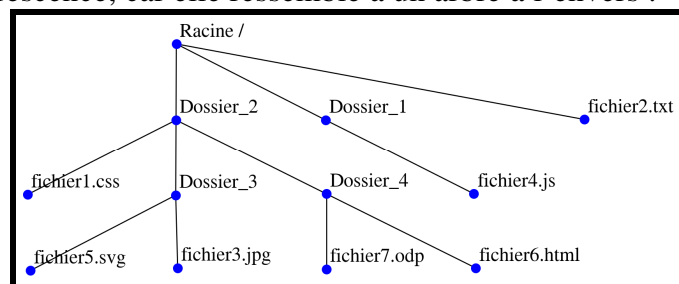
Une URL (Uniform Resource Locator) est l'adresse d'une ressource d'un site Web (par exemple un fichier). Elle indique le protocole, le nom de domaine et le chemin vers la ressource :

http:// www.lyc-turgot.ac-limoges.fr/ spip.php?article2825

Protocole Nom de domaine

Chemin

Elle indique « l'endroit » où se trouve une ressource dans l'arborescence d'un site. Un fichier peut se trouver dans un dossier qui peut lui-même se trouver dans un autre dossier... On parle d'une structure en arborescence, car elle ressemble à un arbre à l'envers :



La base de l'arbre s'appelle la racine de l'arborescence et se représente par un « / » (slash).

2.1. CHEMIN RELATIF ET CHEMIN ABSOLU

Pour indiquer la position d'un fichier (ou d'un dossier) dans l'arborescence, il existe 2 méthodes : indiquer un chemin absolu ou indiquer un chemin relatif.

Le chemin absolu doit indiquer « le chemin » depuis la racine. Par exemple l'URL du fichier « fichier3.jpg » sera : « /Dossier_2/Dossier_3/fichier3.jpg ». Le chemin commence bien de la racine « / » (attention : les symboles de séparation sont aussi des /)

Si le fichier fichier1.css fait appel au fichier fichier3.jpg (comme un fichier HTML peut faire appel à un fichier CSS). Il est possible d'indiquer le chemin non pas depuis la racine, mais depuis le dossier (Dossier_2) qui accueille le fichier1.css, nous parlerons alors de chemin relatif : Dossier_3/fichier3.jpg. On note l'absence du « / » au début du chemin (c'est cela qui nous permettra de distinguer un chemin relatif et un chemin absolu).

Si nous désirions indiquer le chemin relatif du fichier fichier1.css depuis l'intérieur du dossier « Dossier_4 ». Il faut « remonter » d'un « niveau » dans l'arborescence pour se retrouver dans le dossier « Dossier_2 » et ainsi pouvoir repartir vers la bonne « branche ». Pour cela il faut utiliser 2 points : « .. », ainsi : « ../fichier1.css »

Il est tout à fait possible de remonter de plusieurs « niveaux » : « ../../ » depuis le dossier « Dossier_4 » permet de « remonter » à la racine.

Le contenu du fichier « fichier7.odp » utilise le fichier « fichier5.svg ». Donnez le chemin relatif qui devra être renseigné dans le fichier « fichier7.odp » afin d'atteindre le fichier « fichier5.svg ».

Donnez le chemin absolu permettant d'atteindre le fichier « fichier6.html ».

Remarque : La façon d'écrire les chemins (avec des slash (« / ») comme séparateurs) est propre aux systèmes dits « UNIX » (par exemple GNU/Linux), Mac OS ou les chemins réseau (et donc le web). Sous Windows, ce n'est pas le slash qui est utilisé, mais l'antislash (« \ »).

3. HTML, CSS ET JAVASCRIPT

L'HTML, le JavaScript et le CSS sont les trois langages utilisés pour présenter et mettre en forme les documents sur le Web. Ici nous ne ferons qu'une présentation non exhaustive car l'objectif n'est pas de former des spécialistes de ces langages mais de vous permettre de réaliser une page Web simple.

3.1. HTML

HTML (Hypertext Markup Language) est un langage de balisage et non un langage de programmation.

HTML permet de décrire le contenu d'un document et d'en donner la structure.

HTML n'étant pas un langage de programmation, en HTML il n'existe pas de variables, de conditions, fonctions ou autres ...

Sur Wikipédia : L'Hypertext Markup Language, généralement abrégé HTML, est le format de données conçu pour représenter les pages Web. C'est un langage de balisage permettant d'écrire de l'hypertexte, d'où son nom. HTML permet également de structurer sémantiquement et de mettre en forme le contenu des pages, d'inclure des ressources multimédias, dont des images, des formulaires de saisie, et des programmes informatiques. Il permet de créer des documents interopérables avec des équipements très variés de manière conforme aux exigences de l'accessibilité du Web. Il est souvent utilisé conjointement avec des langages de programmation (JavaScript) et des feuilles de style en cascade (CSS). C'est un format ouvert.

3.1.1. HTML, LANGAGE DE BALISAGE

Le principe des balises est d'encapsuler un contenu entre une balise ouvrante et une balise fermante. **Un script correctement indenté est facilement lisible.** Par exemple :

```

<head>
  <title>
    Le titre de la page
  </title>
</head>

```

L'action de ces balises porte alors sur la partie du texte qu'elle contient.

Les balises peuvent être imbriquées mais toujours correctement ordonnées à l'image d'une série de parenthèses et de crochets : [()] ne convient pas, _____ est la bonne série.

Pour écrire du code HTML, il faut être très rigoureux car le navigateur ne renverra pas de message d'erreur pour une mauvaise indentation mais le résultat ne sera pas à la hauteur de vos attentes.

3.1.2. STRUCTURE DU CODE HTML

L'ordre dans lequel sont décrits les balises permet de construire une structure de la page.

Tout en haut, à la racine de cet arbre, on trouve la balise **<html>**. L'ensemble du document est compris entre les balises **<html>** et **</html>**

Dans ce contenu on trouve alors deux parties :

- **<head>** qui contient les informations générales (métadonnées) sur le document (titre, nom de l'onglet, liens vers des scripts et / ou des feuilles de style, ...)
- **<body>** qui contient le contenu principal du document HTML. Un seul par document.

```

<!DOCTYPE html>
<!-- Ceci est un commentaire qui ne s'affichera pas
ATTENTION sauf s'il est entre les balises <title>
<!DOCTYPE html> : indique que le contenu est du code html -->
<html lang="fr" >
<!-- Début du code -->
  <head>
    <!-- Début de l'entête -->
    <meta charset="UTF-8" />
    <!-- Codage UTF-8 pour l'affichage des caractères
    spécifiques -->
    <!-- Début du titre de l'onglet -->
    <title>
      C'est le titre de l'onglet
    <!-- Attention les commentaires sont affichés -->
    </title>
    <!-- Fin du titre de l'onglet -->
  </head>
  <!-- Fin de l'entête -->
  <body>
    <!-- Début du corps de la page -->
    <h1>
      <!-- Titre de la page -->
      C'est le titre de la page
    </h1>

```

```

<!-- Fin du titre -->
<p>
<!-- Paragraphe 1 de la page -->
    C'est le paragraphe 1 de la page
</p>
<p>
<!-- Paragraphe 2 de la page -->
    <!-- br permet de faire un retour à la ligne -->
    C'est le paragraphe 2 de la page qui permet <br> de
    faire le test sur le codage UTF8 avec le mot <br><br><br>
    même <br><br><br> en supprimant meta charset="UTF-8"
</p>
<!-- Fin du paragraphe -->
</body>
<!-- Fin du corps de la page -->
</html>
<!-- Fin du code -->

```

3.1.2.1.LES BALISES, LES ATTRIBUTS ET LES ELEMENTS DU CODE HTML

Les balises ouvrantes sont constituées, au minimum, d'un nom entre chevrons :

```
<head>
```

Vous pouvez trouver la liste des balises sur cette page :

<https://developer.mozilla.org/fr/docs/Web/HTML/Element>

Mais elles peuvent contenir également des attributs (Ils sont définis après le nom de la balise et sont formés d'une paire nomAttribut="valeur" ou nomAttribut='valeur') qui permettent de fournir des informations complémentaires sur l'élément :

```

<link rel="stylesheet" href="essai1Css_lien_html.css" />
<ol type = I>

```

Vous pouvez trouver la liste des attributs sur cette page :

<https://developer.mozilla.org/fr/docs/Web/HTML/Attributs>

Les balises fermantes sont constituées uniquement d'un nom entre chevrons précédé du caractère slash (« / ») :

```
</head>
```

Un élément se compose d'une balise ouvrante, d'un contenu et d'une balise fermante (avec le nom identique à celui de la balise ouvrante):

```

<body>
  <div>
    <h1>
      Ici ce trouve le titre du texte
    </h1>
    <p>
      Ici ce trouve le paragraphe du texte.
    </p>
  </div>
</body>

```

Un élément vide est constitué d'une balise ouvrante et se termine par «/»

```
<meta charset="UTF-8" />
```

```

```

Les balises ont toutes un rôle, dont voici la liste non exhaustive :

Balises	Rôles
<html>	Marque le début du code (représente la racine du document HTML)
<head>	Contient les informations générales (métadonnées) sur le document (titre, nom de l'onglet, liens vers des scripts et / ou des feuilles de style, ...)
<meta>	Contient des informations à propos de la page. On y retrouve les informations à propos des styles, des scripts et des données destinées à aider les moteurs de recherche et les navigateurs. Les métadonnées relatives à la mise en forme ou aux scripts
<title>	Définit le titre du document, il est affiché dans la barre du titre du navigateur ou de l'onglet de la page.
<body>	Représente le contenu principal du document HTML. Il ne peut y avoir qu'un élément <body> par document.
<h1> ...	<h1> à <h6> représentent six niveaux de titres dans un document, <h1> est le plus important et <h6> est le moins important. Un élément de titre décrit brièvement le sujet de la section qu'il introduit.
<p>	Représente un paragraphe de texte. Les paragraphes sont généralement représentés comme des blocs et séparés par un espace vertical, leur première ligne est également parfois indentée. Ce sont des éléments blocs.
<div>	Signifie division du document et c'est un conteneur générique qui permet d'organiser le contenu sans représenter rien de particulier. Il peut être utilisé afin de grouper d'autres éléments.
<link>	Définit la relation entre le document courant et une ressource externe (feuille de style, icônes, ...).
	Permet de représenter une image dans un document.
	Représente une liste d'éléments sans ordre particulier et souvent représenté par une liste à puces.
	Représente une liste ordonnée généralement affichée avec un indicateur ordinal pouvant prendre la forme de nombres, de lettres, de chiffres romains ou de points.
Avec l'attribut type : <ol type = I>	Type = I : numérotation type I, II, III, ... Type = i : numérotation type i, ii, iii, ... Type = A : numérotation type A, B, C, ... Type = a : numérotation type a, b, c, ... Type = 1 : numérotation type 1, 2, 3, ...
Avec l'attribut start : <ol start = 5>	Numérotation à partir du nombre
	Utilisé pour représenter un élément dans une liste. Il doit être contenu dans un élément parent : une liste ordonnée (), une liste non ordonnée () ou un menu (<menu>).
<hr>	Représente un changement thématique entre des éléments de paragraphe.
<audio>	Permet d'intégrer un contenu sonore dans un document. Il peut contenir une ou plusieurs sources audio représentées avec l'attribut src ou l'élément <source>.

Balises	Rôles
<video>	Permet d'intégrer un contenu vidéo dans un document.
<table>	Permet de représenter un tableau de données, c'est-à-dire des informations exprimées sur un tableau en deux dimensions.
<caption>	Représente la légende (ou le titre) d'un tableau. Il doit être le premier élément parmi les descendants de l'élément <table>.
<figcaption>	Représente la légende (ou le titre) d'une photo, d'une vidéo ou d'un son.
<th>	Définit une cellule d'un tableau comme une cellule d'en-tête pour un groupe de cellule. La nature de ce groupe est définie grâce aux attributs <code>scope</code> et <code>headers</code> .
<td>	Définit une cellule d'un tableau qui contient des données.
	Gras
<i>	Italique
<u>	souligné
<a>	Signifie ancre et permet de définir un hyperlien vers un autre endroit de la même page ou vers une autre page sur le Web.

Un élément de type `block` va toujours commencer sur une nouvelle ligne et prendre toute la largeur disponible dans la page.

Un élément de type `block` peut contenir d'autres éléments de type `block` ainsi que des éléments de type `inline`.

Les éléments HTML de type `block` les plus communs sont les suivants `p`, `h1`, `h2`, `ol`, `ul`, `div`, ...

Les éléments de type `inline` ne vont pas commencer sur une nouvelle ligne mais s'insérer dans la ligne actuelle.

Les éléments de type `inline` prennent uniquement la largeur qui leur est nécessaire (c'est-à-dire la largeur de leur contenu).

Les éléments HTML de type `inline` les plus communs sont les suivants `strong`, `em`, `a`, `img`, `span`, ...

Pour plus d'informations sur les éléments HTML consultez les liens suivants :

<https://developer.mozilla.org/fr/docs/Web/HTML/Element>

https://www.w3schools.com/html/html_blocks.asp

3.1.2.2. ENVIRONNEMENT DE DEVELOPPEMENT (IDE) ET VERIFICATION DE LA VALIDITE / CONFORMITE DU CODE HTML

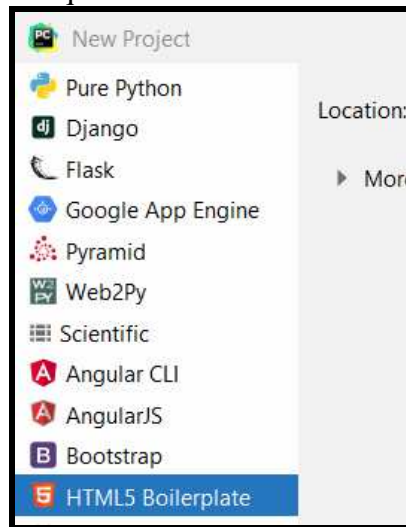
Pour créer une page html, un simple éditeur de texte ou un bloc note suffisent mais il faut sauvegarder le fichier avec l'extension `.html`.

JSFiddle est un des outils qui permettent de développer un site Internet. Il faut ensuite acheter un nom de domaine et héberger le site.

<https://www.000webhost.com/> permet d'héberger gratuitement un site, vous devez vous inscrire puis le dossier `public_html` doit contenir les dossiers nécessaires à la page Web ainsi que le fichier principal nommé `index.html`.

<https://validator.w3.org/> permet de vérifier la validité / la conformité de votre code HTML.

Soit le script précédant pages 3 et 4, testez-le en utilisant Pycharm et en créant un nouveau projet HTML comme indiqué ci-dessous :



Soit le script précédant pages 3 et 4, testez-le en supprimant « `<meta charset="UTF-8"/>` ». Que constatez-vous ?

3.1.2.3.LES ANCRES ET LES LIENS HTML

Il est possible de créer une ancre dans une page, c'est-à-dire de marquer un endroit précis d'une page pour s'y rendre par un lien hypertexte. Les ancres se définissent grâce à l'attribut id.

La syntaxe est la suivante :

```
<div id="Essai-Ancre">
    C'est ici l'essai d'ancre
</div>
```

L'appel se fait ainsi si l'ancre est dans la même page :

```
<a href="#Essai-Ancre">
    Essai ancre
</a>
```

L'appel se fait ainsi si l'ancre est dans une autre page :

```
<a href="url/nom_du_fichier.#Essai-Ancre">
    Essai ancre
</a>
```

Il est également possible de créer un lien hypertexte vers une page Web. La syntaxe est la suivante :

```
<a href="http://www.lyc-turgot.ac-limoges.fr/">
    Site du lycée Turgot
</a>
```

3.1.2.4.LES IMAGES

On utilise l'élément `<img>` pour insérer les images. Deux attributs sont obligatoires :

- L'attribut `src` pour spécifier le nom du fichier image à charger. Les fichiers image doivent impérativement être aux formats GIF, PNG ou JPG.

- L'attribut `alt` pour indiquer un contenu alternatif, c'est à dire un texte à afficher à la place de l'image lorsque, pour différentes raisons, elle n'apparaît pas.

La syntaxe est la suivante :

```

```

Les attributs `height` et `width` permettent de redimensionner l'image.

3.1.2.5. LES VIDEOS

On utilise l'élément `<video>` pour insérer les vidéos.

La syntaxe est la suivante :

```
<video controls width="500">
  <source src="img/rose.mp4" type="video/mp4">
  <source src="img/rose.webm" type="video/webm">
  <source src="img/rose.ogg" type="video/ogg" />
  Impossible de lire la vidéo.
</video>
```

L'attribut `width` permet de redimensionner la vidéo.

L'attribut `src` pour spécifier le nom du fichier vidéo à charger. Ici il y a 3 encodages différents (.mp4, .ogg et .webm).

« Impossible de lire la vidéo. » s'affichera si la lecture est impossible.

Pour plus d'informations sur les vidéos visitez cette page :

<https://www.alsacreations.com/article/lire/1125-introduction-balise-video-html5-mp4-h264-webm-ogg-theora.html>

3.1.2.6. LES SONS

On utilise l'élément `<audio>` pour insérer les enregistrements audios.

La syntaxe est la suivante :

```
<audio controls>
  <source src="img/son.mp3" type="audio/mpeg">
  <source src="img/son.ogg" type="audio/ogg">
  Impossible de lire l'audio.
</audio>
```

L'attribut `src` pour spécifier le nom du fichier image à charger. Ici il y a 2 encodages différents (.mp3 et .ogg).

« Impossible de lire l'audio. » s'affichera si la lecture est impossible.

Afin de modifier la taille du lecteur audio, il faut ajouter ceci dans la balise `<head>` :

```
<style>
  .fenetre_son{
    width: 1200px; height : 200px;
  }
</style>
```

Et appeler la mise en forme ainsi dans la balise `<audio>` :

```
<audio controls class="fenetre_son">
```

3.1.2.7.LES TELECHARGEMENTS

La syntaxe est la suivante :

```
<a href="Apres_NSI.pdf" download="NSI.pdf">TELECHARGER  
LE DOCUMENT EN CLIQUANT ICI</a>
```

L'attribut href indique le document cible avec son extension.

L'attribut download indique le nouveau nom avec son extension donné après téléchargement au document cible.

La syntaxe se termine par l'étiquette apparente sur la page, dans cet exemple :

« TELECHARGER LE DOCUMENT EN CLIQUANT ICI »

3.1.3. EXERCICE HTML

En utilisant :

- Les informations précédentes et les liens suivants :

<https://developer.mozilla.org/fr/docs/Web/HTML/Element>

<https://developer.mozilla.org/fr/docs/Web/HTML/Attributs>

<https://www.alsacreations.com/article/lire/1125-introduction-balise-video-html5-mp4-h264-webm-ogg-theora.html>

https://developer.mozilla.org/fr/docs/Apprendre/HTML/Multimedia_and_embed_ding/Images_in_HTML

<https://developer.mozilla.org/fr/docs/Web/HTML/Element/video>

<https://developer.mozilla.org/fr/docs/Web/HTML/Element/audio>

- Le document, l'image, la vidéo et le son suivants :

Apres_NSI.pdf

walle2.PNG

rose.mp4

son.mp3

Ecrivez le code HTML permettant d'obtenir l'affichage de la page suivante.

Sauvegardez votre travail sur <https://www.000webhost.com/>, vérifiez la validité et la conformité de votre code HTML sur <https://validator.w3.org/> puis visualisez le résultat final :

Un titre (avec font color="pink")

Un paragraphe (avec font color="#8a2be2")

Titre du tableau : Fruit et légumes

	Produits	kcal/100g
Fruits	Abricots	43
	Ananas	55
Légumes	Artichauts	47
	Asperges	15

[Essai ancre au bas de cette page](#)

[Site du lycée Turgot](#)

- Puce 1 niveau 1
- Puce 2 niveau 1
 - Puce 1 niveau 2 (avec strong)
 - Puce 2 niveau 2

A. Liste ordonnée 1 niveau 1 (avec u)
 B. Liste ordonnée 2 niveau 1 (avec b)
 C. Liste ordonnée 3 niveau 1 (avec i)

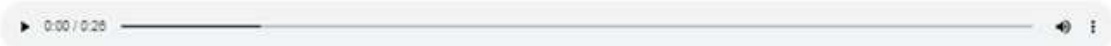
Voir une photo



Voir un film



Ecoutez un son



Télécharger un document

[TELECHARGER LE DOCUMENT EN CLIQUANT ICI](#)

3.2. CSS

Les feuilles de style en cascade, généralement appelées CSS de l'anglais Cascading Style Sheets, forment un langage informatique qui décrit la présentation des documents HTML et XML. Les standards définissant CSS sont publiés par le World Wide Web Consortium (W3C). Introduit au milieu des années 1990, CSS devient couramment utilisé dans la conception de sites Web et bien pris en charge par les navigateurs Web dans les années 2000.

Le langage HTML comme nous l'avons vu sert à implémenter le contenu de façon structurée. Le CSS lui va permettre de rendre tout cela agréable à lire et à consulter. Ce langage vise à séparer la description de la mise en page d'un document (CSS) de sa structure (HTML). Les possibilités sont infinies. Les pièges sont nombreux (trop chargés, illisible en changeant de support, etc.), on parle de **responsiv design** (ou Responsive Web Design (RWD)) qui est une technique de conception d'interface digitale faisant en sorte que l'affichage d'une quelconque page d'un site s'adapte de façon automatique à la taille de l'écran du terminal de lecture). Bootstrap est une collection d'outils utiles à la création du design (graphisme, animation et interactions avec la page dans le navigateur, etc.) de sites et d'applications Web. C'est un

ensemble qui contient des codes HTML et CSS, des formulaires, boutons, outils de navigation et autres éléments interactifs. Il contient des Framework CSS permettent l'obtention de site de bonne qualité relativement facilement.

3.2.1. DU CODE HTML SANS CSS

Afin de voir l'effet du code CSS sur la mise en page d'une page Web, saisissez et commentez le code HTML suivant :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta name="Faye" content="Essai HTML avec images" />
    <title>
      Exemple CSS
    </title>
  </head>
  <body>
    <div>
      <h1>
        Essai
      </h1>
      <h2>
        Du Css et du Html
      </h2>
      <h3>
        Avec &nbsp;
        <span>
          fichier
        </span> &nbsp; Css indépendant
      </h3>
    </div>
    <p>
      Ceci est un paragraphe justifié.
    </p>
    <p>
      Ceci est un autre paragraphe justifié.
    </p>
    <h3>
      
    </h3>
    <h4>
      
    </h4>
    <h5>
      
    </h5>
  </body>
</html>
```

Que constatez vous ?

Quel est le rôle de « » dans ce code HTML ?

3.2.2. DU CODE HTML AVEC CSS

Afin de lier le fichier CSS suivant « TP16Exo4_Css_liier_html.css » au fichier HTML ci-dessus que vous venez de saisir vous ajouter la balise suivante sous la balise fermante </title> :

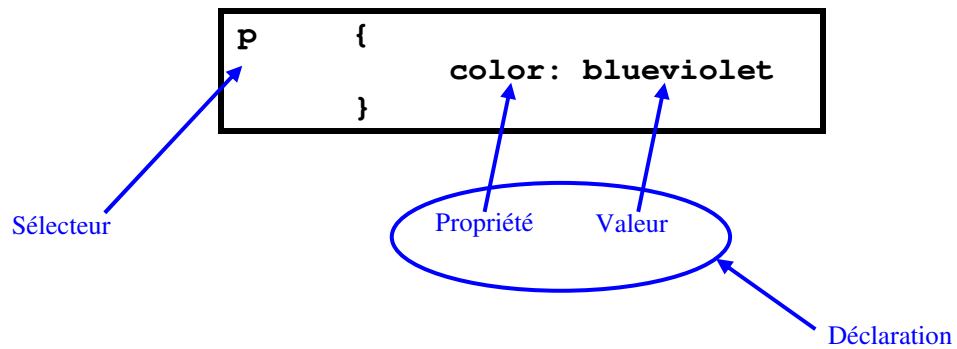
```
<link rel="stylesheet" href="TP16Exo4_Css_liier_html.css" />
```

Vous devez ensuite saisir et commentez (/* entre */) le code CSS suivant dans le fichier « TP16Exo4_Css_liier_html.css » :

```
h1 {
    text-align: center ;
    font-size: large ;
    color: red;
}
p {
    text-align: justify;
    font-size: 24pt;
    font-family: Verdana;
    font-style: italic;
    color: blueviolet;
    font-weight: 900;
}
h2 {
    text-align: center ;
    color: blue;
}
h3 {
    text-align: center ;
    color: black;
}
h2,h3 {
    font-size: xx-large ;
}
h4 {
    text-align : center ;
    border: 10px solid black
}
h5 {
    text-align : left ;
}
h6 {
    text-align : right ;
}
div{
    color: white; /*Textes des éléments dans les div blancs
                  par défaut*/
    background-color: yellowgreen; /*Les div auront un fond
                                    vert*/
    width: 80%; /*Définit la largeur des div à 80% de leur
                parent (body ici)*/
    margin: 0 auto; /*Permet de centrer les div dans leur
                    élément parent (body ici)*/
}
span{
    font-weight: bold; /*Les textes seront en gras*/
    background-color: yellow; /*Fond des span jaune*/
    color: black; /*Couleur du texte noire*/
}
```

Que constatez vous en exécutant le code HTML ?

3.2.3. ANATOMIE D'UNE REGLE CSS



Sélecteur : c'est le nom de l'élément HTML situé au début de l'ensemble de règles. Il permet de sélectionner les éléments sur lesquels appliquer le style souhaité (en l'occurrence, les éléments `p`). Pour mettre en forme un élément différent, il suffit de changer le sélecteur.

Déclaration : c'est une règle simple comme « `color : blueviolet` », qui détermine les propriétés de l'élément que l'on veut mettre en forme.

Propriétés : les différentes façons dont on peut mettre en forme un élément HTML (dans ce cas, « `color` » est une propriété des éléments « `p` »). En CSS, vous choisissez les différentes propriétés que vous voulez utiliser dans une règle CSS.

Valeur de la propriété : à droite de la propriété, après les deux points, on a la valeur de la propriété. Celle-ci permet de choisir une mise en forme parmi d'autres pour une propriété donnée (on peut choisir d'autres couleurs que « `blueviolet` » pour la propriété « `color` »).

Les autres éléments importants de la syntaxe sont :

- chaque ensemble de règles, à l'exception du sélecteur, doit être entre accolades « `{ }` ».
- pour chaque déclaration, il faut utiliser deux points « `:` » pour séparer la propriété de ses valeurs.
- pour chaque ensemble de règles, il faut utiliser un point-virgule « `;` » pour séparer les déclarations entre elles.

Si on veut modifier plusieurs propriétés d'un coup, on peut utiliser plusieurs déclarations dans une seule règle en les séparant par des points-virgules :

```
p { text-align: justify;
    font-size: 24pt;
    font-family: Verdana;
    font-style: italic;
    color: blueviolet;
    font-weight: 900;
}
```

Il est aussi possible de sélectionner plusieurs d'éléments pour appliquer à tous une même règle. Il suffit de placer plusieurs sélecteurs, séparés par des virgules :

```
h2,h3 { font-size: xx-large ;
}
```

Il y a différents types de sélecteurs. Nous n'avons vu que les sélecteurs d'élément qui permettent de sélectionner les éléments HTML d'un type donné dans un document HTML. Il est possible de faire des sélections plus spécifiques :

- Les sélecteurs de balise sélectionnent toutes les balises portant ce nom :

```
p {  
    color: blueviolet  
}
```

- Les sélecteurs d'identifiant « id » sélectionnent **LA** balise portant cet identifiant. Ajoutez, entre la 2^{ème} balise </p> et la balise <h4> du script HTML précédant, le code suivant :

```
<p id="p1">  
    Ceci est un autre paragraphe justifié pour tester l'id="p1" dans  
la feuille de style CSS.  
</p>  
<p id="p2">  
    Ceci est un 4ème paragraphe justifié pour tester l'id="p2" dans  
la feuille de style CSS.  
</p>
```

Ajoutez au script CSS le code suivant :

```
#p1 {  
    color: darkred;  
    background-color: black  
}  
#p2 {  
    color: black;  
    background-color: yellow  
}
```

Que constatez vous en exécutant le code HTML ?

- Les sélecteurs de classes sélectionnent **TOUTES** les balises portant cette classe. Ajoutez, entre la dernière balise </p> et la balise <h4> du script HTML précédant, le code suivant :

```
<p class="p1">  
    Ceci est un autre paragraphe justifié pour tester class="p1"  
dans la feuille de style CSS.  
</p>  
<p class="p2">  
    Ceci est un autre paragraphe justifié pour tester class="p2"  
dans la feuille de style CSS.  
</p>  
<p class="p1">  
    Ceci est un 2ème paragraphe justifié pour tester class="p1" dans  
la feuille de style CSS.  
</p>  
<p class="p2">  
    Ceci est un 2ème paragraphe justifié pour tester class="p2" dans  
la feuille de style CSS.  
</p>
```

Ajoutez au script CSS le code suivant :

```
.p1 { color: yellow;
      background-color: burlywood;
    }
.p2 { color: rosybrown;
      background-color: crimson;
    }
```

Que constatez vous en exécutant le code HTML ?

- La pseudo-classe « hover » permet de spécifier l'apparence d'un élément au moment où l'utilisateur le survole avec le pointeur, sans nécessairement l'activer. Ajoutez, entre la dernière balise </p> et la balise <p class="p1"> du script HTML précédant, le code suivant :

```
<p>
  <a href="#">
    Ce lien sera écrit sur un fond gris lors du survol par
    le pointeur
  </a>
</p>
```

Ajoutez au script CSS le code suivant :

```
}
a:hover { background-color: grey;
}
```

Que constatez vous en exécutant le code HTML ?

3.2.4. L'HERITAGE EN CSS

L'héritage est une notion centrale et fondamentale en CSS.

La notion d'héritage signifie que tout élément HTML « enfant » va hériter, « en cascades », des styles de ses parents.

C'est par ailleurs de là que vient le nom du CSS : Cascading StyleSheets, ou Feuilles de Style en Cascades.

Par exemple, tous les éléments à l'intérieur de l'élément body sont des enfants de cet élément. Si l'on applique un style à l'élément body, ses enfants en hériteront automatiquement.

L'objectif du programme n'est pas faire de vous des spécialistes du CSS et du HTML mais vous pouvez poursuivre vos recherches sur les liens fournis ci-dessus et sur ce lien :

<https://www.pierre-giraud.com/home-old.php>

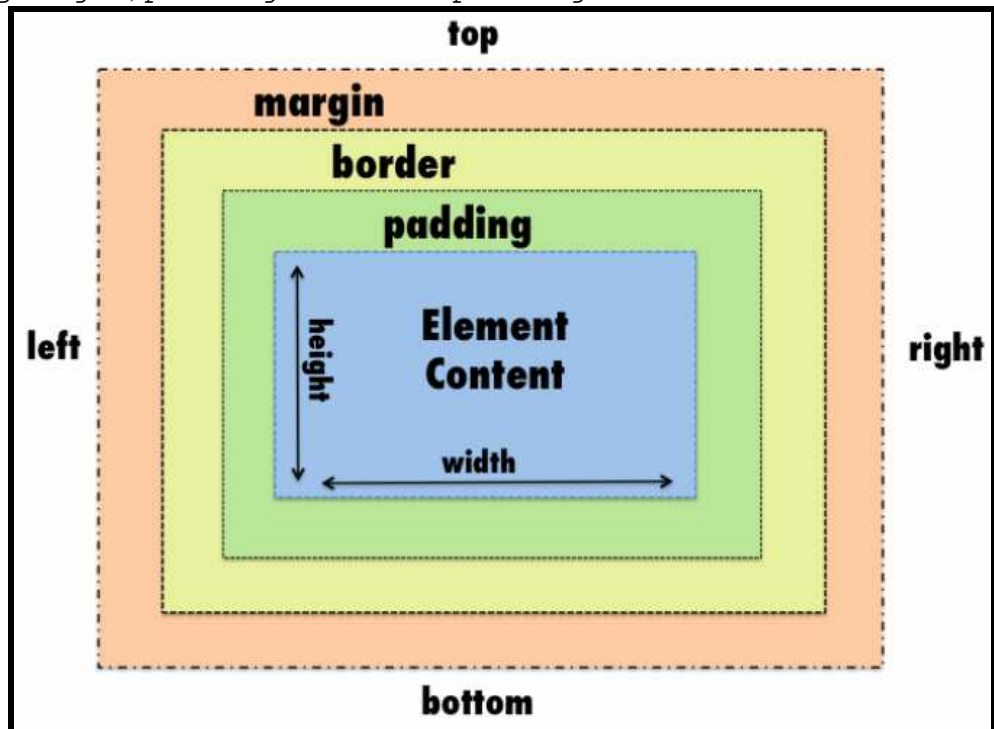
3.2.5. LES UNITES DE MESURE EN CSS

CSS offre différentes unités pour exprimer les dimensions. Certaines proviennent de la typographie, comme le point (pt) et le pica (pc), d'autres sont connues pour leur usage quotidien, comme le centimètre (cm) et le pouce (in). Et il y a également celle inventée spécifiquement pour CSS: le pixel (px). Les unités n'ont rien à voir avec les propriétés, mais avec le média de sortie: écran ou papier. Il n'y a pas de restriction à utiliser telle unité à tel ou tel endroit. Si une propriété accepte une valeur en px (margin: 5px) elle accepte également une valeur en pouces ou en centimètres (margin: 1.2in ou margin: 0.5cm) et vice-versa.

3.2.6. PADDING ET MARGIN

La propriété `margin` définit la taille des marges sur les quatre côtés de l'élément. Elle sont donnée dans le sens horaire en partant du haut. C'est une propriété raccourcie qui permet de manipuler les autres propriétés de marges : `margin-top`, `margin-right`, `margin-bottom` et `margin-left`. Il est possible d'utiliser des valeurs négatives pour chacun des côtés.

La propriété `padding` est une propriété raccourcie qui permet de définir les différents écarts de remplissage sur les quatre côtés d'un élément. Elle synthétise `padding-top`, `padding-right`, `padding-bottom` et `padding-left`.



Afin de visualiser les effets de `margin` et de `padding`, vous pouvez visiter les liens suivants :

<https://developer.mozilla.org/fr/docs/Web/CSS/padding>

<https://developer.mozilla.org/fr/docs/Web/CSS/margin>

Ecrivez les codes HTML et CSS permettant d'obtenir l'affichage suivant :

Fond (couleur purple)

En tête (2000px, couleur #FF9900, padding 50 px partout, margin 50px en haut et 0px ailleurs, centré)

Partie de gauche (750px, couleur #FF3366, padding 10px partout, margin 50px en haut et 0px ailleurs, hauteur min 400px, aligné à gauche)

Partie de droite (margin 50px en haut, 850px à gauche et 0px ailleurs, couleur #9966FF, padding 10px partout, hauteur min 400px, aligné à droite)

Partie principale (2000px, hauteur min 500px, couleur red, padding 50 px partout, margin 50px en haut et 0px ailleurs, centré)

Pied de Page (2000px, couleur #669933, padding 50 px partout, centré)

3.3. JAVASCRIPT

L'objectif de cette partie n'est pas de faire de vous des codeurs de JavaScript experts, mais simplement de vous donner les clés qui vous permettront d'explorer ce langage.

JavaScript ou js est un langage de programmation coté client (tout se passe sur votre machine) à contrario de PHP qui lui est déporté coté serveur.

C'est un langage interprété qui va nous permettre de créer de l'interaction entre le client et le serveur (dialogue client-serveur). Il permet de rendre une page dynamique.

3.3.1. LA PLACE DU JAVASCRIPT

Le JavaScript se place à trois endroits différents mais le placer en fin de fichier accélère le rendu de la page :

- Dans l'élément head d'une page HTML,
- Dans l'élément body d'une page HTML,
- Dans un fichier portant l'extension .js séparé.

3.3.1.1. DANS L'ELEMENT HEAD

Il faudra placer le JavaScript à l'intérieur d'une balise script (dans un élément script). On le placera généralement dans l'élément head d'un code HTML.

```
<script>
    alert("Message pour l'utilisateur !!! script dans le
                                         head) ");
</script>
```

Que constatez vous en exécutant le code HTML contenant le code précédent ?

3.3.1.2. DANS L'ELEMENT BODY

Il faudra placer le JavaScript à l'intérieur d'une balise script (dans un élément script) d'un code HTML..

```
<script>
    alert("Message pour l'utilisateur !!! script dans le
                                         body) ");
</script>
```

Que constatez vous en exécutant le code HTML contenant le code précédent ?

Que constatez vous en déplaçant cette partie de code dans l'élément body du code HTML ?

3.3.1.3. DANS UN FICHIER .JS SEPARÉ

C'est la méthode recommandée car elle permet la séparation des langages et la réutilisation d'un même code JavaScript dans plusieurs fichiers HTML.

Dans ce cas, nous allons devoir lier nos fichiers HTML et JavaScript en utilisant à nouveau un élément script et son attribut src.

Placez ceci dans le body du code HTML, « TP16Exo9_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo9_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo9_html_liier_js.js » :

```
alert("Message pour l'utilisateur !!! (script dans un fichier
                                         Javascript) ")
```

Que constatez vous en exécutant le code HTML contenant le code précédent ?

3.3.2. LES BLOCS, LES IDENTATIONS ET LES COMMENTAIRES

Le JavaScript doit être indenté et commenté (`//` sur une ligne ou `/* */` sur plusieurs lignes). Un bloc en JavaScript est délimité par des accolades `{ }`.

3.3.3. LA CONSOLE

Nous allons travailler avec un fichier HTML de votre choix. Les navigateurs sont dotés de consoles de développement permettant d'entrer des instructions à la volée, de consulter les données en mémoire ou d'explorer les fonctions et variables disponibles. On accède à la console par un menu pour les développeurs, à l'aide des touches de raccourci :

Navigateurs	Raccourcis
Exploreurs	F12 ou « outils » puis « outils de développement »
Firefox	« Menu » puis « développement web » puis « outils de développement »
Chrome	« Menu » puis « plus d'outils » puis « outils de développement »
Edge	F12 ou « outils supplémentaires » puis « outils de développement »
Opéra	Ctrl+Shift+I ou via le « menu » puis « outils de développement » puis « opera dragonfly »
Safari	Activer dans les options avancées, en cochant la case « Afficher le menu développement dans la barre des menus » puis Ctrl+Alt+I ou Ctrl+Alt+C

Placez ceci dans le body du code HTML, « TP16Exo10_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo10_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo10_html_liier_js.js » :

```
alert("Tapez F12 pour voir le message écrit dans la console
      (script dans un fichier JavaScript)");
console.log("Voici le message que j'écris dans la console
            !!!");
```

Que constatez vous en exécutant le code HTML contenant le code précédent ?

Que constatez vous en ouvrant la console de votre navigateur ?

Dans la console, saisissez les instructions suivantes et précisez ce que vous constatez en les exécutant :

```
console.log("Bonjour");
```

Que constatez vous ?

Dans la console, saisissez les instructions suivantes et précisez ce que vous constatez en les exécutant :

```
var fruit = "kiwis";
var mavariable = "J'aime les "
console.log(mavariable+fruit);
```

Que constatez vous ?

Dans la console, saisissez les instructions suivantes et précisez ce que vous constatez en les exécutant :

```
var fruits = [
    { name: "Kiwi", vitamine: "C" },
    { name: "Mangue", vitamine: "A" },
    { name: "Cassis", vitamine: "E" }
];
console.table(fruits);
```

Que constatez vous ?

Sur la page Web d'un site de votre choix et à l'aide de la console de votre navigateur, saisissez l'instruction suivante et précisez ce que vous constatez en l'exécutant :

```
document.querySelector('html').style.backgroundColor='purple';
```

Que constatez vous ?

Sur la page Web d'un site de votre choix et à l'aide de la console de votre navigateur, saisissez l'instruction suivante et précisez ce que vous constatez en l'exécutant :

```
var monImage = document.createElement('img');
monImage.setAttribute('src', 'https://c1.staticflickr.com/1/57
2/20463320350_58483f6bed.jpg');
document.querySelector('h1').appendChild(monImage);
```

Que constatez vous ?

Allez sur la page Web du lycée Turgot, de Google ou d'un site de votre choix et à l'aide de la console de votre navigateur, modifiez le code HTML pour modifier la couleur du texte ou du fond de l'article. Que constatez vous ?

A l'aide de la console de votre navigateur, explorez la constitution de cette page Web.

3.3.4. LES VARIABLES (ET LES CONSTANTES)

Une variable JavaScript est un conteneur servant à stocker temporairement une information. Il faut déclarer les variables. Leurs noms sont sensibles à la casse (majuscule / minuscule).

Il y a trois manières de déclarer une variable en JavaScript : `let`, `const` et `var`.

3.3.4.1. AVEC L'INSTRUCTION « LET »

Elle permet de déclarer une variable globale ou locale. On peut la déclarer sans valeur initiale et on peut la modifier après déclaration.

Placez ceci dans le body du code HTML, « TP16Exo11_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo11_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo11_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
JavaScript)");

let maVar = false;
console.log(maVar);
console.log(typeof(maVar));
let maVar1 = 2;
console.log(maVar1);
console.log(typeof(maVar1));
let maVar2 = 2.3;
console.log(maVar2);
console.log(typeof(maVar2));
let maVar3 = "a";
console.log(maVar3);
console.log(typeof(maVar3));
let maVar4;
console.log(maVar4);
console.log(typeof(maVar4));
maVar4=maVar+maVar3+maVar2;
console.log(maVar4);
console.log(typeof(maVar4));
maVar4=maVar1+maVar2;
console.log(maVar4);
console.log(typeof(maVar4));
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console.

C'est une variable globale si elle est déclarée en début de script :

Placez ceci dans le body du code HTML, « TP16Exo12_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo12_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo12_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
JavaScript)");
let maVar=1;
console.log(maVar);
if (true)
{
    maVar+=2;
    console.log(maVar);
}
maVar+=5;
console.log(maVar);
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console.

C'est une variable locale si elle est déclarée dans un bloc (fonction, for, if, ...) entre accolades {} :

Placez ceci dans le body du code HTML, « TP16Exo13_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo13_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo13_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
                                           JavaScript)");

if (true)
{
    let maVar=2;
    console.log(maVar);
}
console.log("En dehors du if");
maVar+=1;
console.log(maVar);
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console.

Placez ceci dans le body du code HTML, « TP16Exo14_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo14_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo14_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
                                           JavaScript)");

let maVar2 = 2;
console.log("maVar2 vaut avant le bloc :");
console.log(maVar2);
{
    maVar2=maVar2*4;
    console.log("maVar2 vaut dans le bloc :");
    console.log(maVar2);
    let maVar3 = 3;
    console.log("maVar3 vaut dans le bloc :");
    console.log(maVar3);
}
console.log("maVar2 à l'extérieur du bloc :");
console.log(maVar2);
console.log("maVar3 à l'extérieur du bloc :");
console.log(maVar3);
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console.

Modifiez le code pour qu'il affiche les 2 variables sans enlever les accolades.

Placez ceci dans le body du code HTML, « TP16Exo14_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo14bis_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo14bis_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
                                       JavaScript)");
for (let i = 0; i < 5; i++)    {
                               console.log("i dans la boucle
                                       for vaut : ");
                               console.log(i);
                               }
console.log("i en dehors de la boucle for vaut : ");
console.log(i);
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console. Modifiez le code pour qu'il n'affiche pas d'erreur sans enlever les accolades..

3.3.4.2.AVEC L'INSTRUCTION « CONST »

Elle permet de déclarer une constante globale ou locale. Elle doit être déclarée avec une valeur initiale et on ne peut pas la modifier après déclaration lorsqu'elle est un entier, un flottant ou une chaîne de caractère.

Placez ceci dans le body du code HTML, « TP16Exo16_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo16_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo16_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
                                       JavaScript)");

const maVar = false;
console.log(maVar);
console.log(typeof(maVar));
const maVar1 = 2;
console.log(maVar1);
console.log(typeof(maVar1));
const maVar2 = 2.3;
console.log(maVar2);
console.log(typeof(maVar2));
const maVar3 = "a";
console.log(maVar3);
console.log(typeof(maVar3));
const maVar4 = 6;
console.log(maVar4);
console.log(typeof(maVar4));
maVar4=maVar+maVar3+maVar2 ;
console.log(maVar4);
console.log(typeof(maVar4));
maVar4=maVar1+maVar2;
```

```
console.log (maVar4) ;
console.log (typeof (maVar4)) ;
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console. Modifiez le code pour qu'il affiche toutes les constantes.

On peut la modifier une variable créée avec l'instruction « const » après déclaration lorsqu'elle est un tableau.

Placez ceci dans le body du code HTML, « TP16Exo18_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo18_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo18_html_liier_js.js » :

```
alert ("Tapez F12 pour voir la console (script dans un fichier
                                JavaScript)");

const tabl = [];
tabl.push (1);
tabl.push (2);
tabl.push ("Le tableau augmente");
console.log (tabl) ; ;
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console. Que fait la méthode « push » ?

3.3.4.3.AVEC L'INSTRUCTION « VAR »

L'instruction var est à éviter depuis la version ES6 = ES2015.

« ES » est l'abréviation d'ECMAScript, le standard sur lequel repose JavaScript.

La mise à jour du standard JavaScript est désormais annuelle. Afin de mieux s'y repérer, vous verrez donc également une appellation basée sur l'année :

ES6 = ES2015

ES7 = ES2016

ES8 = ES2017

ES9 = ES2018

ES10 = ES2019

ES11 = ES2020

ES12 = ES2021 ...

3.3.4.4.EXERCICE

Ecrire un fichier HTML avec un fichier JavaScript séparé permettant d'afficher la valeur de $r = x^2 + 2yz$. Il faudra déclarer quatre variables x , y , z et r et afficher le résultat r à l'aide de la fonction `alert`. Pour demander les valeurs à l'utilisateur, vous devrez utiliser les fonctions `parseInt` ou `parseFloat` et `prompt` (voir encadré ci-dessous). Les variables x et z sont entières et les variables y et r sont décimales.

```
x = parseInt (prompt ("x", "Saisir la valeur entière de x ici")) ;
```

3.3.5. LES FONCTIONS

Une fonction est introduite par le mot clé `function` suivi des arguments, puis du code de la fonction entre accolades `{ }`. Le résultat de la fonction est indiqué par le mot-clé `return`. La déclaration peut se faire à l'intérieur de l'élément `head` dans un élément `script` ou bien directement dans un fichier JavaScript séparé. Mais la déclaration ne doit pas se faire à l'intérieur de l'élément `body`.

Cette procédure permet de déclarer une fonction, il faut ensuite afficher le résultat :

```
document.write("Message à afficher", nom de la variable à  
afficher);
```

Ecrivez le code HTML et JavaScript permettant d'obtenir l'affichage de la page ci-dessous. Vous devrez pour y parvenir :

- Ecrire une fonction en JavaScript qui à une valeur x renvoie x^2 .
 - Réaliser un tableau de valeur de la fonction précédente de 0 à 4 avec un pas de 1. (utilisez la balise `table`)
 - Vous pouvez incorporer du code JavaScript dans du code html.
 - Vous pouvez incorporer du code html dans votre code JavaScript.
- Tester le code suivant :

```
document.write("<h1>Ceci est un titre de niveau 1 en HTML  
dans le script du Javascript</h1>")
```

- Vous pouvez faire une boucle `for` en JavaScript (exemple page 22).

Tableau de valeur de la fonction carrée					
x	0	1	2	3	4
x ²	0	1	4	9	16

Dans le body, le tableau est :
0,0,1,1,2,4,3,9,4,16

3.3.6. LA METHODE PROMPT()

La méthode `prompt()` fournit un moyen simple de récupérer une information provenant de l'utilisateur, on parle alors de boîte de dialogue invites. La méthode `prompt()` requiert deux arguments :

- le texte d'invite
- la chaîne de caractères par défaut dans le champ de saisie

Placez ceci dans le `body` du code HTML, « `TP16Exo21_html_liier_js.js` » étant le nom du fichier JavaScript :

```
<script src="TP16Exo21_html_liier_js.js">  
</script>
```

Et placez ceci dans le fichier JavaScript « `TP16Exo21_html_liier_js.js` » :

```
function demande() {  
    x=prompt("Quel âge as tu?", "Saisie ici  
ton âge");  
    return x;  
}  
document.write("Tu as ");  
document.write(demande());  
document.write(" ans, tu es très très vieux");
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ?

Réalisez une fonction JavaScript, qui lise un nombre au clavier en utilisant la méthode `prompt()`. Qui calcule le carré de la valeur saisie en utilisant la fonction `Math.pow()` vu ci-dessus puis qui affiche le carré du nombre saisi ainsi : « Le carré du nombre 9 est 81 ».

3.3.7. LES EVENEMENTS INTERACTION HOMME / MACHINE

Les événements sont des actions de l'utilisateur, qui vont pouvoir donner lieu à une interactivité. L'événement par excellence est le clic de souris, car c'est le seul que le HTML gère. Grâce au JavaScript il est possible d'associer des fonctions, des méthodes à des événements tels que le passage de la souris au-dessus d'une zone, le changement d'une valeur, ...

La syntaxe d'un gestionnaire d'événement est la suivante :

```
onEvenement="Action_Javascript_ou_Fonction();" 
```

Pour plus d'informations vous pouvez consulter les sites suivants :

<https://www.w3schools.com/jsref/event onclick.asp>

<https://www.w3schools.com/jsref/event ondragenter.asp>

Placez ceci dans le body du code HTML :

```
<p id="prem" onClick="laFonction()">Cliquez ici (sélecteur  
                                d'identification : prem)</p>  
<p id="deux" onClick="laFonction2()">Ou cliquez là (selecteur  
                                d'identification : deux)</p>  
<p onclick="alert('Vous avez cliqué sur le troisième !!!');">  
    Ou cliquez ici (ouvrir une fenêtre "alerte")</p>  
<p id="traîne" onDragLeave="laFonction3()">Sélectionnez et  
    déplacez (sélecteur d'identification : traîne)</p>  
<p id="passage" onMouseOver="laFonction4()">Passez la souris  
    (sélecteur d'identification : passage)</p>  
<script>  
    function laFonction() {  
        document.getElementById("prem").innerHTML = "Vous  
        avez cliqué sur le premier !!! (sélecteur  
        d'identification : prem)";  
    }  
    function laFonction2() {  
        document.getElementById("deux").innerHTML = "Vous  
        avez cliqué sur le deuxième (sélecteur  
        d'identification : deux)!!!";  
    }  
    function laFonction3() {  
        document.getElementById("traîne").innerHTML =  
        "Vous avez sélectionné et déplacé !!! (sélecteur  
        d'identification : traîne)";  
    }  
</script>
```

```

        function laFonction4() {
            document.getElementById("passage").innerHTML =
                "Vous avez passé la souris !!! (sélecteur
                d'identification : passage)";
        }
    </script>

```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Testez d'autres évènements.

3.3.7.1. LA METHODE QUERYSELECTOR()

La méthode `querySelector()` sélectionne le premier élément dans le document correspondant au sélecteur (ou groupe de sélecteurs) spécifié, ou `null` si aucune correspondance n'est trouvée.

Pour plus d'informations vous pouvez consulter les sites suivants :

https://www.w3schools.com/jsref/met_document_queryselector.asp

<https://developer.mozilla.org/fr/docs/Web/API/Document/querySelector>

https://www.w3schools.com/jsref/met_document_queryselectorall.asp

Placez ceci dans le body du code HTML :

```

<p onClick="laFonction()">
    Cliquez ici (sans sélecteur, donc s'applique au premier
                                élément)
</p>
<p class="classe" onClick="laFonction2()">
    Ou cliquez là (sélecteur de classe "classe")
</p>
<p id="ident" onClick="laFonction3()">
    Ou cliquez ici (sélecteur d'identifiant' "ident")
</p>
<div>
    <p>
        Première division
    </p>
</div>
<div>
    <p>
        Deuxième division
    </p>
</div>
<p onClick="laFonction4()">
    Ou cliquez encore ici (sans sélecteur, donc s'applique à
    la première division)
</p>
<p>
    'C'est l'histoire d'un garçon troublé ...
</p>
<p style="color: white" class="suite">
    ... car il s'avère que son ours en peluche a disparu ...
    <br>

```

```
<br>
... mais finalement il le retrouve ...
<br>
<br>
... et c'est tant mieux.' '
</p>
<p onClick="laFonction5()">
    Cliquez ici pour connaître la suite de l'histoire
    (sélecteur de classe "suite")
</p>
</body>
<script>
    function laFonction() {
        document.querySelector("p").style.color =
            "red";
    }
    function laFonction2() {
        document.querySelector("p.classe").
            style.backgroundColor = "yellow";
    }
    function laFonction3() {
        document.querySelector("#ident").
            style.color = "blue";
    }
    function laFonction4() {
        document.querySelector(("div > p")).style.
            backgroundColor = "green";
    }
    function laFonction5() {
        document.querySelector("p.suite").style.
            backgroundColor = "black";
    }
</script>
</body>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Testez d'autres exemples.

Vous pouvez remarquer que s'il y a plusieurs paragraphes seul le premier élément dans le sélecteur p est modifié car la méthode `querySelector()` ne s'applique sur le premier sélecteur de la page. Pour remédier à cela il faut donner un sélecteur d'identifiant id ou un sélecteur de classe class au paragraphe concerné.

3.3.7.2.LES BOUTONS

Nous allons intégrer à notre page des boutons.

Placez ceci dans le body du code HTML :

```
<h3>
    Les boutons
</h3>
<p>
```

```

    Cliquez sur les boutons ci dessous :
</p>
<button type="button" id="bout" onclick="laFonction()">
    Cliquez ici
</button>
<br>
<button onclick="quitte()">
    Fermer la fenêtre
</button>
<br>
<button onclick="laFonction2()">
    Ou cliquez ici
</button>
<script>
    function laFonction() {
        let x = document.getElementById("bout");
        x.disabled = true;
    }
    function laFonction2() {
        let x = document.createElement("button");
        let t = document.createTextNode("Cliquez
                                          moi");
        x.appendChild(t);
        document.body.appendChild(x);
    }
    function quitte() {
        if (confirm("Veux tu vraiment fermer
                    cette fenêtre ?"))
        {
            alert("A bientôt alors !");
            close();
        }
    }
</script>

```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Testez d'autres exemples.

3.3.7.3. LES FORMULAIRES

Un formulaire HTML est entre les balises `<form>` et `</form>` et contient notamment des champs de saisie appelés `input` de différents types ou `select`.

Les attributs `value` seront des variables utilisées en JavaScript coté client ou côté serveur (PHP).

3.3.7.3.1. INTERACTION AVEC LES CHAMPS DE SAISIE INPUT

Il existe différents types de champs de saisie `input` : `tel`, `search`, `url`, `email`, `number`, `range`, `color`, `date`, `time`, `text`, ...

Pour plus d'informations vous pouvez consulter le site suivant :

<https://developer.mozilla.org/fr/docs/Web/HTML/Element/Input>

Placez ceci dans le body du code HTML :

```

<form>
    <p>Login : <input type="text" name="lgn"/></p>

```

```

<p>Mot de passe : <input type="password" name="mdp"/></p>
<p>Catégorie :
    Femme <input type="radio" name="sexe" value="fem"/>
    Homme <input type="radio" name="sexe" value="hom"/>
</p>
<p>Spécialité(s) préférée(s) : </br>
    NSI <input type="checkbox" name="inf" value="info"/>
    Maths <input type="checkbox" name="mat"
                                value="math"/>
    Autres <input type="checkbox" name="aut"
                                value="autr"/>
</p>
<p><input type="button" value="Cliquez ici" id="bout"
                                onclick="laFonction()"></p>
<p><input type="submit" value="Envoyer"></p>
</form>
<script>
    function laFonction() {
        let x = document.getElementById("bout");
        x.disabled = true;
    }
</script>

```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Testez d'autres types de champs de saisie input.

Remarques :

- Pour que les boutons soient mutuellement exclusifs, ils doivent avoir le même attribut name : dans l'exemple précédent : "sexe".
- le terme boutons « radio » vient des anciennes radio. On pouvait mémoriser les stations de radio et les sélectionner à l'aide de gros boutons munis de ressorts. Lorsqu'on enfonçait un bouton, un mécanisme ressortait le bouton qui avait été enfoncé auparavant : les boutons étaient ainsi mutuellement exclusifs.



source : <https://st2.depositphotos.com>

- Plutôt que d'utiliser une balise input de type bouton :

```

<p><input type="button" value="Cliquez ici" id="bout"
                                onclick="laFonction()">
</p>

```

On peut utiliser balise button (vu au paragraphe précédent) qui permet d'obtenir le même rendu visuel :

```

</p>
<button type="button" id="bout" onclick="laFonction()">
    Cliquez ici
</button>

```

3.3.7.3.2. INTERACTION AVEC LES CHAMPS SELECT

La liste déroulante select fait parti des éléments interactifs importants.

Pour plus d'informations vous pouvez consulter le site suivant :

https://www.w3schools.com/jsref/dom_obj_select.asp

Placez ceci dans le body du code HTML :

```
<h1>
  Sélectionnez votre spécialité
</h1>
<form>
  <select name="Spécialités">
    <option value="nsi">Numérique et sciences
informatiques</option>
    <option value="maths">Mathématiques</option>
    <option value="autres">Autres spécialités</option>
  </select>
</form>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Testez d'autres méthodes et d'autres propriétés de la liste déroulante select.

La liste déroulante select permet un seul choix par défaut mais on peut autoriser un choix multiple. Dans ce cas les valeurs sélectionnées sont traitées comme un tableau.

Placez ceci dans le body du code HTML :

```
<h1>
  Sélectionnez vos spécialités
</h1>
<h3>
  Pour faire des choix multiples, sélectionnez vos choix en
appuyant sur "ctrl"
</h3>
<form>
  <select name="Spécialités[]" multiple>
    <option value="nsi">Numérique et sciences
informatiques</option>
    <option value="maths">Mathématiques</option>
    <option value="autres">Autres spécialités</option>
  </select>
</form>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ?

Il est possible de sélectionner par défaut un (ou plusieurs éléments) en ajoutant l'attribut selected à la balise <option>.

Placez ceci dans le body du code HTML :

```
<h1>
  Sélectionnez vos spécialités
```

```

</h1>
<h3>
    Pour faire des choix multiples, sélectionnez vos choix en
    appuyant sur "ctrl"
</h3>
<form>
    <select name="Spécialités[]" multiple>
        <option value="nsi" selected> Numérique et sciences
            informatiques</option>
        <option value="maths"> Mathématiques</option>
        <option value="autres" selected> Autres
            spécialités</option>
    </select>
</form>

```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ?

3.3.7.3.3. EXERCICE

Ecrivez le code HTML et JavaScript permettant d'obtenir l'affichage de la page ci-dessous. Vous devrez pour y parvenir :

- utiliser les balises input et select,
- préciser le type lors de l'utilisation d'une balise input :

Affichage	Type de l'input
Nom	text
e-mail	email
Mot de passe	passwd
Numéro	text
Validité	date
Bouton « Valider le paiement »	submit

- préciser le name dans chacun des cas (attribut name utilisé plus tard pour retrouver la valeur d'un élément d'un formulaire).
- proposer trois modèles de carte bancaire : « Carte Visa », « Carte Bleue » (sélectionnée par défaut) et « Carte Mastercard ».
- le bouton « Valider le paiement » active une fonction du script Javascript. Cette fonction doit demander au client confirmation au message : « Voulez vous vraiment valider les renseignements ? ». Si le client confirme le message précédant alors le message suivant s'affiche « C'est validé !!! » sinon l'affichage est « Ce n'est pas validé ».

Formulaire de paiement
 Les champs obligatoires sont précédés par "*"

Informations de contact
 Qualité : Monsieur ☐ Madame ☐
 Nom : *
 e-mail : *
 Mot de passe : *

Informations de paiement
 Type de carte :
 Numéro : *
 Validité : *

 format de la validité: jj/mm/aaaa

3.3.7.4. RECUPERATION DES VALEURS DES FORMULAIRES

Nous allons récupérer les valeurs d'un formulaire et en faire l'usage au niveau client.
Placez ceci dans le body du code HTML :

```
<form>
  <p>
    Choix d'une nationalité :
  </p>
  <h6>
    Pensez à visualiser la console en appuyant sur F12
  </h6>
  <select id="choix" name="pays" onchange="selection()">
    <option value="ras">Choisir sa nationalité</option>
    <option value="fr">Français</option>
    <option value="cn">Chinois</option>
    <option value="it">Italien</option>
  </select>
  <p>
    Vous avez choisi comme nationalité :
    <span id="nat">
    </span>
  </p>
</form>
<script>
  function selection() {
    const selecteur = document.getElementById('choix');
    const monChoix = selecteur[selecteur.selectedIndex];
    // visualisation dans la console
    console.log(monChoix.index+' '+monChoix.value +' '+
                                                         monChoix.text);
    document.getElementById('nat').innerHTML =
                                                         monChoix.text;
  }
</script>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Que se passe-t-il dans la console (F12) ?

Dans le script précédent, grâce à la gestion des événements faite par `onchange`, chaque changement d'option lance la fonction `selection()`.

`getElementById('id')` est une méthode qui récupère l'objet de la page identifié par le sélecteur d'identifiant `'id'`. Donc `document.getElementById('choix')` récupère l'objet de la page en cours identifié par `'choix'` :

index	Value	text
0	ras	Choisir sa nationalité
1	fr	Français
2	cn	Chinois
3	it	Italien

`selectedIndex` est une méthode qui renvoie la valeur de l'option choisie dans une liste déroulante. C'est-à-dire que cette méthode indique le rang (de 0 à ...) de l'élément de la liste qui a été sélectionnée par l'utilisateur (dans notre cas de 0 à 3).

`selecteur.selectedIndex` renvoie l'indice du choix fait par l'utilisateur dans la liste déroulante nommée « `selecteur` » (dans notre cas un nombre entre 0 et 3 en fonction du choix fait).

`innerHTML` est une méthode qui permet de récupérer tout le contenu HTML d'un élément d'une page html.

`document.getElementById('nat').innerHTML` intègre le contenu HTML à l'emplacement de la page identifié par le sélecteur d'identifiant nommé 'nat' (dans notre cas une des 4 textes en fonction du choix fait).

3.3.7.4.1. EXERCICE 1

Ecrire un formulaire qui demande votre âge et qui indique dans la même page si vous êtes majeur ou mineur.

3.3.7.4.2. EXERCICE 2

Placez ceci dans le body du code HTML :

```
<form>
  <fieldset>
    <legend>Veuillez sélectionner les spécialités que
              vous conserverez l'année prochaine :
    </legend>
    <div>
      <input type="checkbox" id="nsi" name="interest"
              value="nsi">
      <label for="nsi">NSI</label>
    </div>
    <div>
      <input type="checkbox" id="ma" name="interest"
              value="ma">
      <label for="ma">Maths</label>
    </div>
    <div>
      <input type="checkbox" id="si" name="interest"
              value="si">
      <label for="si">SI</label>
    </div>
  </fieldset>
</form>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Ajoutez une spécialité supplémentaire à cette liste à cocher.

En vous inspirant de l'exercice précédent, affichez la ou les réponses dans la page HTML. Si la réponse contient NSI, la page HTML doit afficher : « Vous conservez NSI, bravo ceci est un très bon choix ! ». Dans les autres cas la page HTML doit afficher : « Vous conservez nom de la matière ».

4. INTERACTION CLIENT - SERVEUR HTTP

La structure client - serveur désigne un mode de communication entre programmes :

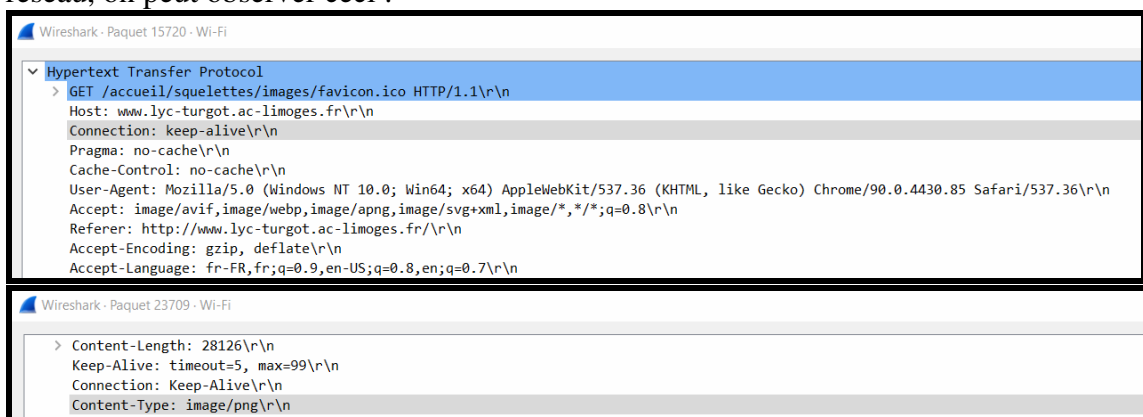
- l'un qualifié de client, envoie des requêtes,
- l'autre, le serveur, y répond.

4.1. PROTOCOLE HTTP

Dans le cas du Web, le client est le navigateur de l'ordinateur utilisé, le serveur est un ordinateur du réseau Internet (on ne connaît pas sa position géographique) et le protocole utilisé est HTTP.

Le protocole HTTP(Hyper Text Transfer Protocol) est un protocole du Web d'échange entre un client et un serveur Web

En utilisant Wireshark, logiciel permettant d'observer les échanges qui transitent sur un réseau, on peut observer ceci :



- User-Agent : donne le client et la version utilisé pour ce protocole, Chrome dans ce cas.
- GET : la méthode utilisée.
- Host : URL (Uniform Resource Locator) adresse Web, site du lycée dans ce cas.
- Content-type : le type de fichier renvoyé par le serveur, un .png dans ce cas.

Les différentes méthodes applicables au protocole HTTP sont GET, HEAD, POST, OPTIONS, CONNECT, TRACE, PUT, PATCH, DELETE

- GET : demande des informations et la ressource désignée. Les paramètres de la demande sont transmissent dans l'URL.
- POST : C'est une méthode qui permet de soumettre des données en vue d'un traitement (côté serveur). C'est cette méthode que nous utiliserons pour envoyer au serveur les données issues d'un formulaire.
- HEAD : demande des informations concernant la ressource

4.2. PROTOCOLE HTTP SECURISE

Le souci du protocole HTTP est qu'il est transparent (on peut observer et récupérer les informations qui circulent). Ceci pose des problèmes de sécurité évidents (échanges bancaires, vie privée...). La réponse à ce problème est le protocole HTTPS (S est pour Secure).

Le protocole HTTPS se compose de 2 protocoles:

- Le protocole HTTP
- Le protocole de sécurisation des échanges sur Internet SSL / TLS. Ce protocole va créer une sorte de canal sécurisé entre le client et le serveur. SSL (Secure Socket Layer) comme son nom l'indique est une couche supplémentaire sécurisée (actuellement obsolète car vulnérable aux

attaques Man In The Middle (MITM) rendant possible l'interception). Le protocole utilisé est le TLS (Transport Layer Security).

Grâce un échange de clés entre eux, le serveur et le client vont établir une connexion chiffrée dont eux seuls pourront lire le contenu. Car seul le client et le serveur en possession de la clé de décryptage pourront déchiffrer les données reçues. Pour plus d'informations consultez :

<https://www.youtube.com/watch?v=7W7WPMX7arI>

4.3. INTERACTION CLIENT - SERVEUR METHODES GET ET POST

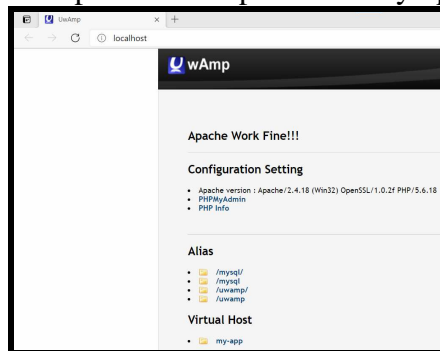
Afin de comprendre l'interaction client – serveur, nous allons utiliser le logiciel permettant de simuler un serveur : Uwamp. Tous les fichiers de travail devront se trouver dans le répertoire « my-app ».



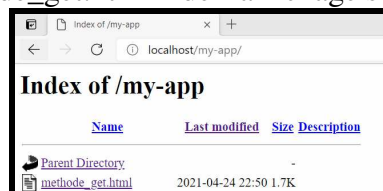
4.3.1. INTERACTION CLIENT - SERVEUR METHODE GET

Cliquez sur « Dossier www » puis mettez dans le répertoire « my-app » les fichiers « methode_get.html » et « reponse_methode_get.php ». Fermez l'explorateur de fichiers.

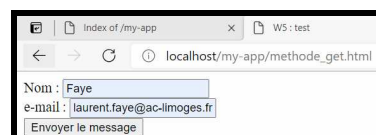
Cliquez sur « Navigateur www » puis sur le répertoire « my-app » de l'affichage suivant :



Cliquez sur le fichier « methode_get.html » de l'affichage suivant :



Complétez les champs suivants et validez en cliquant sur le bouton « Envoyer le message » :



Recopier les informations contenues dans le champ « URL » du navigateur :

Que constatez-vous dans l’affichage de la page Web et dans le champ « URL » du navigateur ?

Où se trouvent les informations transférées par ce formulaire ?

4.3.2. EXERCICE METHODE GET

Reprenez les fichiers précédents et modifier le formulaire pour faire un formulaire qui ressemble à l’affichage suivant (au démarrage NSI est cochée) :



Exercice Méthode Get

localhost/my-app/m...

Veuillez sélectionner les spécialités que vous conserverez l'année prochaine :

☒ NSI

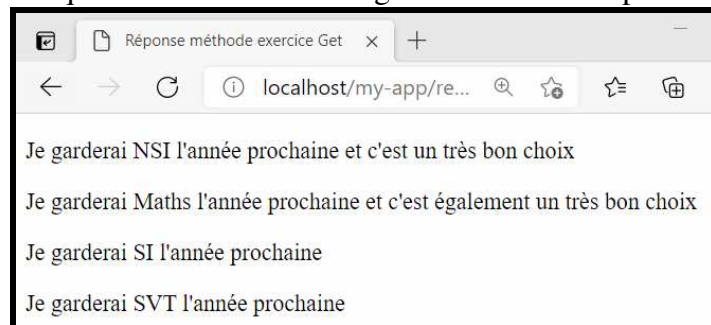
☐ Maths

☐ SI

☐ SVT

Cliquez ici pour envoyer votre choix

Réalisez une réponse qui ressemble à l’affichage suivant si les 4 spécialités sont cochées :



Réponse méthode exercice Get

localhost/my-app/re...

Je garderai NSI l'année prochaine et c'est un très bon choix

Je garderai Maths l'année prochaine et c'est également un très bon choix

Je garderai SI l'année prochaine

Je garderai SVT l'année prochaine

Si aucune spécialité a été cochée, votre réponse ne doit afficher aucune ligne, si une spécialité est cochée une ligne est affichée, si deux spécialités sont cochées deux lignes sont affichées, ... Utilisez une boucle if en PHP et la fonction « empty ».

4.3.3. INTERACTION CLIENT - SERVEUR METHODE POST

Pour utiliser la méthode **POST** dans un formulaire, il suffit de remplacer l’argument **method="GET"** de la balise **form** par **method="POST"**. Il faut également remplacer le nom du fichier réponse dans l’argument **action="reponse_methode_get.php"** par **action="reponse_methode_post.php"**. Modifiez les balises **form** et **title** du fichier « methode_get.html » puis sauvegardez le sous « methode_post.html ». Modifiez les balises **title** et **body** (en remplaçant **\$_GET[user_name]** par **\$_POST[user_name]** et **\$_POST[user_mail]** par **\$_POST[user_mail]**) du fichier « reponse_methode_get.php » puis sauvegardez le sous « reponse_methode_post.php ».

Avec UwAmp, cliquez sur « Dossier www » puis mettez dans le répertoire « my-app » les fichiers « methode_post.html » et « reponse_methode_post.php ». Fermez l’explorateur de fichiers. Cliquez sur « Navigateur www » puis sur le répertoire « my-app ». Cliquez sur le fichier « methode_post.html ». Complétez les champs et validez en cliquant sur le bouton « Envoyer le message ».

Recopier les informations contenues dans le champ « URL » du navigateur :

Que constatez-vous dans l’affichage de la page Web et dans le champ « URL » du navigateur ?

Afin de visualiser les informations transférées par ce formulaire, nous allons accéder à l’outil réseau de votre navigateur (voir page 18 de ce cours) avec F12 ou « outils » puis « outils de développement ». Exécutez le fichier « methode_post.html » sous UwAmp, accédez à l’outil réseau de votre navigateur, actualisez la page Web avec . Dans l’outil réseau, sélectionnez « reponse_methode_post.php » puis « données du formulaire ».

Quelles sont les informations transférées par ce formulaire ?

Quelle différence observez vous entre les méthodes « GET » et « POST »?

La méthode « GET » est à choisir si on veut pouvoir partager une adresse (moteur de recherche par exemple). On peut choisir la méthode post quand on ne veut pas que URL soit modifiée.

4.3.4. EXERCICE METHODE POST

Modifiez les fichiers html et php l’exercice 4.3.2 pour qu’ils fonctionnent avec la méthode « POST ».

5. TRAVAIL A FAIRE

Votre travail est de réaliser un script HTML, CSS, JavaScript commenté que vous intégrerez dans votre site permettant de faire le bilan du travail fait cette année sous la forme d’un carnet de bord.

Vous pouvez réaliser des photos, des sons ou des vidéos de deux minutes maximums qui montrent votre travail (code et effet). Pour y parvenir, prenez appui sur ce cours et fouillez sur Internet. Vous pouvez y intégrer les idées suivantes :

- Ecrire des fonctions qui donnent :
 - le nombre de clics sur la page Web,
 - le temps passé sur la page Web,
 - Le nombre de visite sur la page Web, ...
- Cacher du texte et le faire apparaître avec un clic (pour une solution d'exercice),
- Cacher du texte dans du texte (pour alléger une page en supprimant les définitions des termes), ...

INTERACTIONS ENTRE L'HOMME ET LA MACHINE AVEC UNE APPLICATION WEB

1. LE WEB

Le « World Wide Web », plus communément appelé « Web » a été développé au CERN (Conseil Européen pour la Recherche Nucléaire) par le Britannique Sir Timothy John Berners-Lee et le Belge Robert Cailliau au début des années 90. A cette époque les principaux centres de recherche mondiaux étaient déjà connectés les uns aux autres, mais pour faciliter les échanges d'information Tim Berners-Lee met au point le système hypertexte. Le système hypertexte permet, à partir d'un document, de consulter d'autres documents en cliquant sur des mots clés. Ces mots « cliquables » sont appelés hyperliens et sont souvent soulignés et en bleu. Ces hyperliens sont plutôt connus aujourd'hui sous le simple terme de « liens ».

Cette première page Web est toujours consultable à l'adresse suivante :

<http://info.cern.ch/hypertext/WWW/TheProject.html>

Tim Berners-Lee a développé le premier navigateur web (logiciel permettant de lire des pages contenant des hypertextes), il l'a appelé « WorldWideWeb ». Il faudra attendre 1993 et l'arrivée du navigateur web « NCSA Mosaic » pour que le web commence à devenir populaire en dehors du monde de la recherche.

Techniquement le web se base sur trois technologies : le protocole HTTP (HyperText Transfert Protocol), les URL (Uniform Resource Locator) et le langage de description HTML (HyperText Markup Language).

Beaucoup de personnes confondent « Web » et « Internet » mais les deux choses n'ont rien à voir. Le « Web » désigne un ensemble de données reliées entre elles par des liens hypertextes et accessible sur Internet. « Internet » est un « réseau de réseau » s'appuyant sur le protocole IP (Internet Protocol) et sur lequel on trouve les applications comme le Web, les emails avec le protocole SMTP (Simple Mail Transfert Protocol) et les transferts de fichiers avec le protocole FTP (File Transfert Protocol).

2. L'URL

Dans la barre d'adresse de votre navigateur web vous trouverez, quand vous visitez un site, des choses de ce type : « <http://www.lyc-turgot.ac-limoges.fr/spip.php?article2825> ».

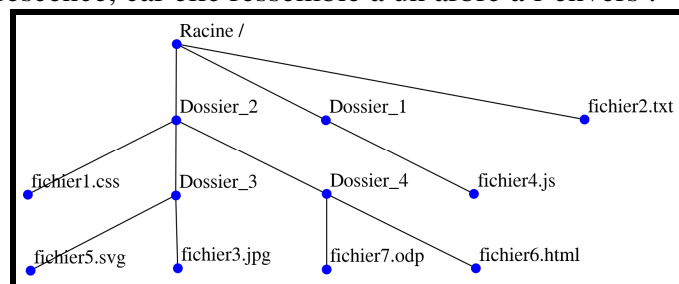
Une URL (Uniform Resource Locator) est l'adresse d'une ressource d'un site Web (par exemple un fichier). Elle indique le protocole, le nom de domaine et le chemin vers la ressource :

http:// www.lyc-turgot.ac-limoges.fr/ spip.php?article2825

Protocole Nom de domaine

Chemin

Elle indique « l'endroit » où se trouve une ressource dans l'arborescence d'un site. Un fichier peut se trouver dans un dossier qui peut lui-même se trouver dans un autre dossier... On parle d'une structure en arborescence, car elle ressemble à un arbre à l'envers :



La base de l'arbre s'appelle la racine de l'arborescence et se représente par un « / » (slash).

2.1. CHEMIN RELATIF ET CHEMIN ABSOLU

Pour indiquer la position d'un fichier (ou d'un dossier) dans l'arborescence, il existe 2 méthodes : indiquer un chemin absolu ou indiquer un chemin relatif.

Le chemin absolu doit indiquer « le chemin » depuis la racine. Par exemple l'URL du fichier « fichier3.jpg » sera : « /Dossier_2/Dossier_3/fichier3.jpg ». Le chemin commence bien de la racine « / » (attention : les symboles de séparation sont aussi des /)

Si le fichier fichier1.css fait appel au fichier fichier3.jpg (comme un fichier HTML peut faire appel à un fichier CSS). Il est possible d'indiquer le chemin non pas depuis la racine, mais depuis le dossier (Dossier_2) qui accueille le fichier1.css, nous parlerons alors de chemin relatif : Dossier_3/fichier3.jpg. On note l'absence du « / » au début du chemin (c'est cela qui nous permettra de distinguer un chemin relatif et un chemin absolu).

Si nous désirions indiquer le chemin relatif du fichier fichier1.css depuis l'intérieur du dossier « Dossier_4 ». Il faut « remonter » d'un « niveau » dans l'arborescence pour se retrouver dans le dossier « Dossier_2 » et ainsi pouvoir repartir vers la bonne « branche ». Pour cela il faut utiliser 2 points : « .. », ainsi : « ../fichier1.css »

Il est tout à fait possible de remonter de plusieurs « niveaux » : « ../../ » depuis le dossier « Dossier_4 » permet de « remonter » à la racine.

Le contenu du fichier « fichier7.odp » utilise le fichier « fichier5.svg ». Donnez le chemin relatif qui devra être renseigné dans le fichier « fichier7.odp » afin d'atteindre le fichier « fichier5.svg ».

Donnez le chemin absolu permettant d'atteindre le fichier « fichier6.html ».

Remarque : La façon d'écrire les chemins (avec des slash (« / ») comme séparateurs) est propre aux systèmes dits « UNIX » (par exemple GNU/Linux), Mac OS ou les chemins réseau (et donc le web). Sous Windows, ce n'est pas le slash qui est utilisé, mais l'antislash (« \ »).

3. HTML, CSS ET JAVASCRIPT

L'HTML, le JavaScript et le CSS sont les trois langages utilisés pour présenter et mettre en forme les documents sur le Web. Ici nous ne ferons qu'une présentation non exhaustive car l'objectif n'est pas de former des spécialistes de ces langages mais de vous permettre de réaliser une page Web simple.

3.1. HTML

HTML (Hypertext Markup Language) est un langage de balisage et non un langage de programmation.

HTML permet de décrire le contenu d'un document et d'en donner la structure.

HTML n'étant pas un langage de programmation, en HTML il n'existe pas de variables, de conditions, fonctions ou autres ...

Sur Wikipédia : L'Hypertext Markup Language, généralement abrégé HTML, est le format de données conçu pour représenter les pages Web. C'est un langage de balisage permettant d'écrire de l'hypertexte, d'où son nom. HTML permet également de structurer sémantiquement et de mettre en forme le contenu des pages, d'inclure des ressources multimédias, dont des images, des formulaires de saisie, et des programmes informatiques. Il permet de créer des documents interopérables avec des équipements très variés de manière conforme aux exigences de l'accessibilité du Web. Il est souvent utilisé conjointement avec des langages de programmation (JavaScript) et des feuilles de style en cascade (CSS). C'est un format ouvert.

3.1.1. HTML, LANGAGE DE BALISAGE

Le principe des balises est d'encapsuler un contenu entre une balise ouvrante et une balise fermante. **Un script correctement indenté est facilement lisible.** Par exemple :

```
<head>
  <title>
    Le titre de la page
  </title>
</head>
```

L'action de ces balises porte alors sur la partie du texte qu'elle contient.

Les balises peuvent être imbriquées mais toujours correctement ordonnées à l'image d'une série de parenthèses et de crochets : [()] ne convient pas, _____ est la bonne série.

Pour écrire du code HTML, il faut être très rigoureux car le navigateur ne renverra pas de message d'erreur pour une mauvaise indentation mais le résultat ne sera pas à la hauteur de vos attentes.

3.1.2. STRUCTURE DU CODE HTML

L'ordre dans lequel sont décrits les balises permet de construire une structure de la page.

Tout en haut, à la racine de cet arbre, on trouve la balise **<html>**. L'ensemble du document est compris entre les balises **<html>** et **</html>**

Dans ce contenu on trouve alors deux parties :

- **<head>** qui contient les informations générales (métadonnées) sur le document (titre, nom de l'onglet, liens vers des scripts et / ou des feuilles de style, ...)
- **<body>** qui contient le contenu principal du document HTML. Un seul par document.

```
<!DOCTYPE html>
<!-- Ceci est un commentaire qui ne s'affichera pas
ATTENTION sauf s'il est entre les balises <title>
<!DOCTYPE html> : indique que le contenu est du code html -->
<html lang="fr" >
<!-- Début du code -->
  <head>
    <!-- Début de l'entête -->
    <meta charset="UTF-8" />
    <!-- Codage UTF-8 pour l'affichage des caractères
    spécifiques -->
    <!-- Début du titre de l'onglet -->
    <title>
      C'est le titre de l'onglet
    <!-- Attention les commentaires sont affichés -->
    </title>
    <!-- Fin du titre de l'onglet -->
  </head>
  <!-- Fin de l'entête -->
  <body>
    <!-- Début du corps de la page -->
    <h1>
      <!-- Titre de la page -->
      C'est le titre de la page
    </h1>
```

```

<!-- Fin du titre -->
<p>
<!-- Paragraphe 1 de la page -->
    C'est le paragraphe 1 de la page
</p>
<p>
<!-- Paragraphe 2 de la page -->
    <!-- br permet de faire un retour à la ligne -->
    C'est le paragraphe 2 de la page qui permet <br> de
    faire le test sur le codage UTF8 avec le mot <br><br><br>
    même <br><br><br> en supprimant meta charset="UTF-8"
</p>
<!-- Fin du paragraphe -->
</body>
<!-- Fin du corps de la page -->
</html>
<!-- Fin du code -->

```

3.1.2.1.LES BALISES, LES ATTRIBUTS ET LES ELEMENTS DU CODE HTML

Les balises ouvrantes sont constituées, au minimum, d'un nom entre chevrons :

```
<head>
```

Vous pouvez trouver la liste des balises sur cette page :

<https://developer.mozilla.org/fr/docs/Web/HTML/Element>

Mais elles peuvent contenir également des attributs (Ils sont définis après le nom de la balise et sont formés d'une paire nomAttribut="valeur" ou nomAttribut='valeur') qui permettent de fournir des informations complémentaires sur l'élément :

```

<link rel="stylesheet" href="essai1Css_lien_html.css" />
<ol type = I>

```

Vous pouvez trouver la liste des attributs sur cette page :

<https://developer.mozilla.org/fr/docs/Web/HTML/Attributs>

Les balises fermantes sont constituées uniquement d'un nom entre chevrons précédé du caractère slash (« / ») :

```
</head>
```

Un élément se compose d'une balise ouvrante, d'un contenu et d'une balise fermante (avec le nom identique à celui de la balise ouvrante):

```

<body>
  <div>
    <h1>
      Ici ce trouve le titre du texte
    </h1>
    <p>
      Ici ce trouve le paragraphe du texte.
    </p>
  </div>
</body>

```

Un élément vide est constitué d'une balise ouvrante et se termine par «/»

```
<meta charset="UTF-8" />
```

```

```

Les balises ont toutes un rôle, dont voici la liste non exhaustive :

Balises	Rôles
<html>	Marque le début du code (représente la racine du document HTML)
<head>	Contient les informations générales (métadonnées) sur le document (titre, nom de l'onglet, liens vers des scripts et / ou des feuilles de style, ...)
<meta>	Contient des informations à propos de la page. On y retrouve les informations à propos des styles, des scripts et des données destinées à aider les moteurs de recherche et les navigateurs. Les métadonnées relatives à la mise en forme ou aux scripts
<title>	Définit le titre du document, il est affiché dans la barre du titre du navigateur ou de l'onglet de la page.
<body>	Représente le contenu principal du document HTML. Il ne peut y avoir qu'un élément <body> par document.
<h1> ...	<h1> à <h6> représentent six niveaux de titres dans un document, <h1> est le plus important et <h6> est le moins important. Un élément de titre décrit brièvement le sujet de la section qu'il introduit.
<p>	Représente un paragraphe de texte. Les paragraphes sont généralement représentés comme des blocs et séparés par un espace vertical, leur première ligne est également parfois indentée. Ce sont des éléments blocs.
<div>	Signifie division du document et c'est un conteneur générique qui permet d'organiser le contenu sans représenter rien de particulier. Il peut être utilisé afin de grouper d'autres éléments.
<link>	Définit la relation entre le document courant et une ressource externe (feuille de style, icônes, ...).
	Permet de représenter une image dans un document.
	Représente une liste d'éléments sans ordre particulier et souvent représenté par une liste à puces.
	Représente une liste ordonnée généralement affichée avec un indicateur ordinal pouvant prendre la forme de nombres, de lettres, de chiffres romains ou de points.
Avec l'attribut type : <ol type = I>	Type = I : numérotation type I, II, III, ... Type = i : numérotation type i, ii, iii, ... Type = A : numérotation type A, B, C, ... Type = a : numérotation type a, b, c, ... Type = 1 : numérotation type 1, 2, 3, ...
Avec l'attribut start : <ol start = 5>	Numérotation à partir du nombre
	Utilisé pour représenter un élément dans une liste. Il doit être contenu dans un élément parent : une liste ordonnée (), une liste non ordonnée () ou un menu (<menu>).
<hr>	Représente un changement thématique entre des éléments de paragraphe.
<audio>	Permet d'intégrer un contenu sonore dans un document. Il peut contenir une ou plusieurs sources audio représentées avec l'attribut src ou l'élément <source>.

Balises	Rôles
<video>	Permet d'intégrer un contenu vidéo dans un document.
<table>	Permet de représenter un tableau de données, c'est-à-dire des informations exprimées sur un tableau en deux dimensions.
<caption>	Représente la légende (ou le titre) d'un tableau. Il doit être le premier élément parmi les descendants de l'élément <table>.
<figcaption>	Représente la légende (ou le titre) d'une photo, d'une vidéo ou d'un son.
<th>	Définit une cellule d'un tableau comme une cellule d'en-tête pour un groupe de cellule. La nature de ce groupe est définie grâce aux attributs <code>scope</code> et <code>headers</code> .
<td>	Définit une cellule d'un tableau qui contient des données.
	Gras
<i>	Italique
<u>	souligné
<a>	Signifie ancre et permet de définir un hyperlien vers un autre endroit de la même page ou vers une autre page sur le Web.

Un élément de type `block` va toujours commencer sur une nouvelle ligne et prendre toute la largeur disponible dans la page.

Un élément de type `block` peut contenir d'autres éléments de type `block` ainsi que des éléments de type `inline`.

Les éléments HTML de type `block` les plus communs sont les suivants `p`, `h1`, `h2`, `ol`, `ul`, `div`, ...

Les éléments de type `inline` ne vont pas commencer sur une nouvelle ligne mais s'insérer dans la ligne actuelle.

Les éléments de type `inline` prennent uniquement la largeur qui leur est nécessaire (c'est-à-dire la largeur de leur contenu).

Les éléments HTML de type `inline` les plus communs sont les suivants `strong`, `em`, `a`, `img`, `span`, ...

Pour plus d'informations sur les éléments HTML consultez les liens suivants :

<https://developer.mozilla.org/fr/docs/Web/HTML/Element>

https://www.w3schools.com/html/html_blocks.asp

3.1.2.2. ENVIRONNEMENT DE DEVELOPPEMENT (IDE) ET VERIFICATION DE LA VALIDITE / CONFORMITE DU CODE HTML

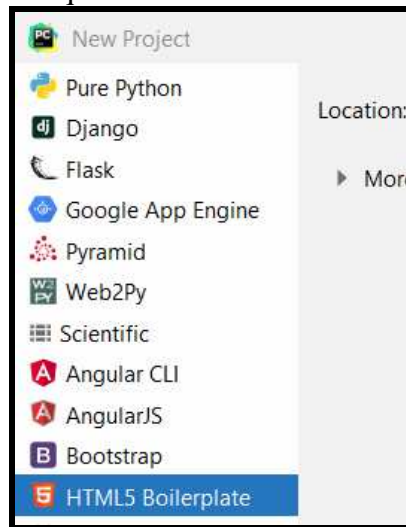
Pour créer une page html, un simple éditeur de texte ou un bloc note suffisent mais il faut sauvegarder le fichier avec l'extension `.html`.

JSFiddle est un des outils qui permettent de développer un site Internet. Il faut ensuite acheter un nom de domaine et héberger le site.

<https://www.000webhost.com/> permet d'héberger gratuitement un site, vous devez vous inscrire puis le dossier `public_html` doit contenir les dossiers nécessaires à la page Web ainsi que le fichier principal nommé `index.html`.

<https://validator.w3.org/> permet de vérifier la validité / la conformité de votre code HTML.

Soit le script précédant pages 3 et 4, testez-le en utilisant Pycharm et en créant un nouveau projet HTML comme indiqué ci-dessous :



Soit le script précédant pages 3 et 4, testez-le en supprimant « `<meta charset="UTF-8"/>` ». Que constatez-vous ?

3.1.2.3.LES ANCRES ET LES LIENS HTML

Il est possible de créer une ancre dans une page, c'est-à-dire de marquer un endroit précis d'une page pour s'y rendre par un lien hypertexte. Les ancres se définissent grâce à l'attribut id.

La syntaxe est la suivante :

```
<div id="Essai-Ancre">
    C'est ici l'essai d'ancre
</div>
```

L'appel se fait ainsi si l'ancre est dans la même page :

```
<a href="#Essai-Ancre">
    Essai ancre
</a>
```

L'appel se fait ainsi si l'ancre est dans une autre page :

```
<a href="url/nom_du_fichier.#Essai-Ancre">
    Essai ancre
</a>
```

Il est également possible de créer un lien hypertexte vers une page Web. La syntaxe est la suivante :

```
<a href="http://www.lyc-turgot.ac-limoges.fr/">
    Site du lycée Turgot
</a>
```

3.1.2.4.LES IMAGES

On utilise l'élément `<img>` pour insérer les images. Deux attributs sont obligatoires :

- L'attribut `src` pour spécifier le nom du fichier image à charger. Les fichiers image doivent impérativement être aux formats GIF, PNG ou JPG.

- L'attribut `alt` pour indiquer un contenu alternatif, c'est à dire un texte à afficher à la place de l'image lorsque, pour différentes raisons, elle n'apparaît pas.

La syntaxe est la suivante :

```

```

Les attributs `height` et `width` permettent de redimensionner l'image.

3.1.2.5. LES VIDEOS

On utilise l'élément `<video>` pour insérer les vidéos.

La syntaxe est la suivante :

```
<video controls width="500">
  <source src="img/rose.mp4" type="video/mp4">
  <source src="img/rose.webm" type="video/webm">
  <source src="img/rose.ogg" type="video/ogg" />
  Impossible de lire la vidéo.
</video>
```

L'attribut `width` permet de redimensionner la vidéo.

L'attribut `src` pour spécifier le nom du fichier vidéo à charger. Ici il y a 3 encodages différents (.mp4, .ogg et .webm).

« Impossible de lire la vidéo. » s'affichera si la lecture est impossible.

Pour plus d'informations sur les vidéos visitez cette page :

<https://www.alsacreations.com/article/lire/1125-introduction-balise-video-html5-mp4-h264-webm-ogg-theora.html>

3.1.2.6. LES SONS

On utilise l'élément `<audio>` pour insérer les enregistrements audios.

La syntaxe est la suivante :

```
<audio controls>
  <source src="img/son.mp3" type="audio/mpeg">
  <source src="img/son.ogg" type="audio/ogg">
  Impossible de lire l'audio.
</audio>
```

L'attribut `src` pour spécifier le nom du fichier image à charger. Ici il y a 2 encodages différents (.mp3 et .ogg).

« Impossible de lire l'audio. » s'affichera si la lecture est impossible.

Afin de modifier la taille du lecteur audio, il faut ajouter ceci dans la balise `<head>` :

```
<style>
  .fenetre_son{
    width: 1200px; height : 200px;
  }
</style>
```

Et appeler la mise en forme ainsi dans la balise `<audio>` :

```
<audio controls class="fenetre_son">
```

3.1.2.7.LES TELECHARGEMENTS

La syntaxe est la suivante :

```
<a href="Apres_NSI.pdf" download="NSI.pdf">TELECHARGER  
LE DOCUMENT EN CLIQUANT ICI</a>
```

L'attribut href indique le document cible avec son extension.

L'attribut download indique le nouveau nom avec son extension donné après téléchargement au document cible.

La syntaxe se termine par l'étiquette apparente sur la page, dans cet exemple :

« TELECHARGER LE DOCUMENT EN CLIQUANT ICI »

3.1.3. EXERCICE HTML

En utilisant :

- Les informations précédentes et les liens suivants :

<https://developer.mozilla.org/fr/docs/Web/HTML/Element>

<https://developer.mozilla.org/fr/docs/Web/HTML/Attributs>

<https://www.alsacreations.com/article/lire/1125-introduction-balise-video-html5-mp4-h264-webm-ogg-theora.html>

https://developer.mozilla.org/fr/docs/Apprendre/HTML/Multimedia_and_embed_ding/Images_in_HTML

<https://developer.mozilla.org/fr/docs/Web/HTML/Element/video>

<https://developer.mozilla.org/fr/docs/Web/HTML/Element/audio>

- Le document, l'image, la vidéo et le son suivants :

Apres_NSI.pdf

walle2.PNG

rose.mp4

son.mp3

Ecrivez le code HTML permettant d'obtenir l'affichage de la page suivante.

Sauvegardez votre travail sur <https://www.000webhost.com/>, vérifiez la validité et la conformité de votre code HTML sur <https://validator.w3.org/> puis visualisez le résultat final :

Un titre (avec font color="pink")

Un paragraphe (avec font color="#8a2be2")

Titre du tableau : Fruit et légumes

	Produits	kcal/100g
Fruits	Abricots	43
	Ananas	55
Légumes	Artichauts	47
	Asperges	15

[Essai ancre au bas de cette page](#)

[Site du lycée Turgot](#)

- Puce 1 niveau 1
- Puce 2 niveau 1
 - Puce 1 niveau 2 (avec strong)
 - Puce 2 niveau 2

A. Liste ordonnée 1 niveau 1 (avec u)
 B. Liste ordonnée 2 niveau 1 (avec b)
 C. Liste ordonnée 3 niveau 1 (avec i)

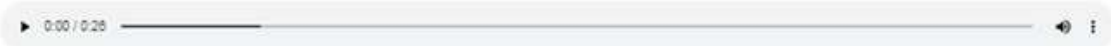
Voir une photo



Voir un film



Ecoutez un son



Télécharger un document

[TELECHARGER LE DOCUMENT EN CLIQUANT ICI](#)

3.2. CSS

Les feuilles de style en cascade, généralement appelées CSS de l'anglais Cascading Style Sheets, forment un langage informatique qui décrit la présentation des documents HTML et XML. Les standards définissant CSS sont publiés par le World Wide Web Consortium (W3C). Introduit au milieu des années 1990, CSS devient couramment utilisé dans la conception de sites Web et bien pris en charge par les navigateurs Web dans les années 2000.

Le langage HTML comme nous l'avons vu sert à implémenter le contenu de façon structuré. Le CSS lui va permettre de rendre tout cela agréable à lire et à consulter. Ce langage vise à séparer la description de la mise en page d'un document (CSS) de sa structure (HTML). Les possibilités sont infinies. Les pièges sont nombreux (trop chargés, illisible en changeant de support, etc.), on parle de **responsiv design** (ou Responsive Web Design (RWD)) qui est une technique de conception d'interface digitale faisant en sorte que l'affichage d'une quelconque page d'un site s'adapte de façon automatique à la taille de l'écran du terminal de lecture). Bootstrap est une collection d'outils utiles à la création du design (graphisme, animation et interactions avec la page dans le navigateur, etc.) de sites et d'applications Web. C'est un

ensemble qui contient des codes HTML et CSS, des formulaires, boutons, outils de navigation et autres éléments interactifs. Il contient des Framework CSS permettent l'obtention de site de bonne qualité relativement facilement.

3.2.1. DU CODE HTML SANS CSS

Afin de voir l'effet du code CSS sur la mise en page d'une page Web, saisissez et commentez le code HTML suivant :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <meta name="Faye" content="Essai HTML avec images" />
    <title>
      Exemple CSS
    </title>
  </head>
  <body>
    <div>
      <h1>
        Essai
      </h1>
      <h2>
        Du Css et du Html
      </h2>
      <h3>
        Avec &nbsp;
        <span>
          fichier
        </span> &nbsp; Css indépendant
      </h3>
    </div>
    <p>
      Ceci est un paragraphe justifié.
    </p>
    <p>
      Ceci est un autre paragraphe justifié.
    </p>
    <h3>
      
    </h3>
    <h4>
      
    </h4>
    <h5>
      
    </h5>
  </body>
</html>
```

Que constatez vous ?

Quel est le rôle de « » dans ce code HTML ?

3.2.2. DU CODE HTML AVEC CSS

Afin de lier le fichier CSS suivant « TP16Exo4_Css_liier_html.css » au fichier HTML ci-dessus que vous venez de saisir vous ajouter la balise suivante sous la balise fermante </title> :

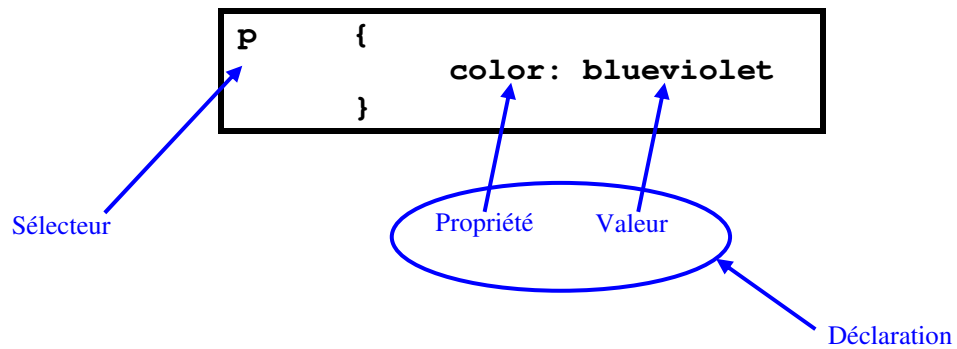
```
<link rel="stylesheet" href="TP16Exo4_Css_liier_html.css" />
```

Vous devez ensuite saisir et commentez (/* entre */) le code CSS suivant dans le fichier « TP16Exo4_Css_liier_html.css » :

```
h1 {
    text-align: center ;
    font-size: large ;
    color: red;
}
p {
    text-align: justify;
    font-size: 24pt;
    font-family: Verdana;
    font-style: italic;
    color: blueviolet;
    font-weight: 900;
}
h2 {
    text-align: center ;
    color: blue;
}
h3 {
    text-align: center ;
    color: black;
}
h2,h3 {
    font-size: xx-large ;
}
h4 {
    text-align : center ;
    border: 10px solid black
}
h5 {
    text-align : left ;
}
h6 {
    text-align : right ;
}
div{
    color: white; /*Textes des éléments dans les div blancs
                  par défaut*/
    background-color: yellowgreen; /*Les div auront un fond
                                    vert*/
    width: 80%; /*Définit la largeur des div à 80% de leur
                parent (body ici)*/
    margin: 0 auto; /*Permet de centrer les div dans leur
                    élément parent (body ici)*/
}
span{
    font-weight: bold; /*Les textes seront en gras*/
    background-color: yellow; /*Fond des span jaune*/
    color: black; /*Couleur du texte noire*/
}
```

Que constatez vous en exécutant le code HTML ?

3.2.3. ANATOMIE D'UNE REGLE CSS



Sélecteur : c'est le nom de l'élément HTML situé au début de l'ensemble de règles. Il permet de sélectionner les éléments sur lesquels appliquer le style souhaité (en l'occurrence, les éléments `p`). Pour mettre en forme un élément différent, il suffit de changer le sélecteur.

Déclaration : c'est une règle simple comme « `color : blueviolet` », qui détermine les propriétés de l'élément que l'on veut mettre en forme.

Propriétés : les différentes façons dont on peut mettre en forme un élément HTML (dans ce cas, « `color` » est une propriété des éléments « `p` »). En CSS, vous choisissez les différentes propriétés que vous voulez utiliser dans une règle CSS.

Valeur de la propriété : à droite de la propriété, après les deux points, on a la valeur de la propriété. Celle-ci permet de choisir une mise en forme parmi d'autres pour une propriété donnée (on peut choisir d'autres couleurs que « `blueviolet` » pour la propriété « `color` »).

Les autres éléments importants de la syntaxe sont :

- chaque ensemble de règles, à l'exception du sélecteur, doit être entre accolades « `{ }` ».
- pour chaque déclaration, il faut utiliser deux points « `:` » pour séparer la propriété de ses valeurs.
- pour chaque ensemble de règles, il faut utiliser un point-virgule « `;` » pour séparer les déclarations entre elles.

Si on veut modifier plusieurs propriétés d'un coup, on peut utiliser plusieurs déclarations dans une seule règle en les séparant par des points-virgules :

```
p { text-align: justify;
    font-size: 24pt;
    font-family: Verdana;
    font-style: italic;
    color: blueviolet;
    font-weight: 900;
}
```

Il est aussi possible de sélectionner plusieurs d'éléments pour appliquer à tous une même règle. Il suffit de placer plusieurs sélecteurs, séparés par des virgules :

```
h2,h3 { font-size: xx-large ;
}
```

Il y a différents types de sélecteurs. Nous n'avons vu que les sélecteurs d'élément qui permettent de sélectionner les éléments HTML d'un type donné dans un document HTML. Il est possible de faire des sélections plus spécifiques :

- Les sélecteurs de balise sélectionnent toutes les balises portant ce nom :

```
p {  
    color: blueviolet  
}
```

- Les sélecteurs d'identifiant « id » sélectionnent **LA** balise portant cet identifiant. Ajoutez, entre la 2^{ème} balise </p> et la balise <h4> du script HTML précédant, le code suivant :

```
<p id="p1">  
    Ceci est un autre paragraphe justifié pour tester l'id="p1" dans  
la feuille de style CSS.  
</p>  
<p id="p2">  
    Ceci est un 4ème paragraphe justifié pour tester l'id="p2" dans  
la feuille de style CSS.  
</p>
```

Ajoutez au script CSS le code suivant :

```
#p1 {  
    color: darkred;  
    background-color: black  
}  
#p2 {  
    color: black;  
    background-color: yellow  
}
```

Que constatez vous en exécutant le code HTML ?

- Les sélecteurs de classes sélectionnent **TOUTES** les balises portant cette classe. Ajoutez, entre la dernière balise </p> et la balise <h4> du script HTML précédant, le code suivant :

```
<p class="p1">  
    Ceci est un autre paragraphe justifié pour tester class="p1"  
dans la feuille de style CSS.  
</p>  
<p class="p2">  
    Ceci est un autre paragraphe justifié pour tester class="p2"  
dans la feuille de style CSS.  
</p>  
<p class="p1">  
    Ceci est un 2ème paragraphe justifié pour tester class="p1" dans  
la feuille de style CSS.  
</p>  
<p class="p2">  
    Ceci est un 2ème paragraphe justifié pour tester class="p2" dans  
la feuille de style CSS.  
</p>
```

Ajoutez au script CSS le code suivant :

```
.p1 { color: yellow;
      background-color: burlywood;
    }
.p2 { color: rosybrown;
      background-color: crimson;
    }
```

Que constatez vous en exécutant le code HTML ?

- La pseudo-classe « hover » permet de spécifier l'apparence d'un élément au moment où l'utilisateur le survole avec le pointeur, sans nécessairement l'activer. Ajoutez, entre la dernière balise </p> et la balise <p class="p1"> du script HTML précédant, le code suivant :

```
<p>
  <a href="#">
    Ce lien sera écrit sur un fond gris lors du survol par
    le pointeur
  </a>
</p>
```

Ajoutez au script CSS le code suivant :

```
}
a:hover { background-color: grey;
          }
```

Que constatez vous en exécutant le code HTML ?

3.2.4. L'HERITAGE EN CSS

L'héritage est une notion centrale et fondamentale en CSS.

La notion d'héritage signifie que tout élément HTML « enfant » va hériter, « en cascades », des styles de ses parents.

C'est par ailleurs de là que vient le nom du CSS : Cascading StyleSheets, ou Feuilles de Style en Cascades.

Par exemple, tous les éléments à l'intérieur de l'élément body sont des enfants de cet élément. Si l'on applique un style à l'élément body, ses enfants en hériteront automatiquement.

L'objectif du programme n'est pas faire de vous des spécialistes du CSS et du HTML mais vous pouvez poursuivre vos recherches sur les liens fournis ci-dessus et sur ce lien :

<https://www.pierre-giraud.com/home-old.php>

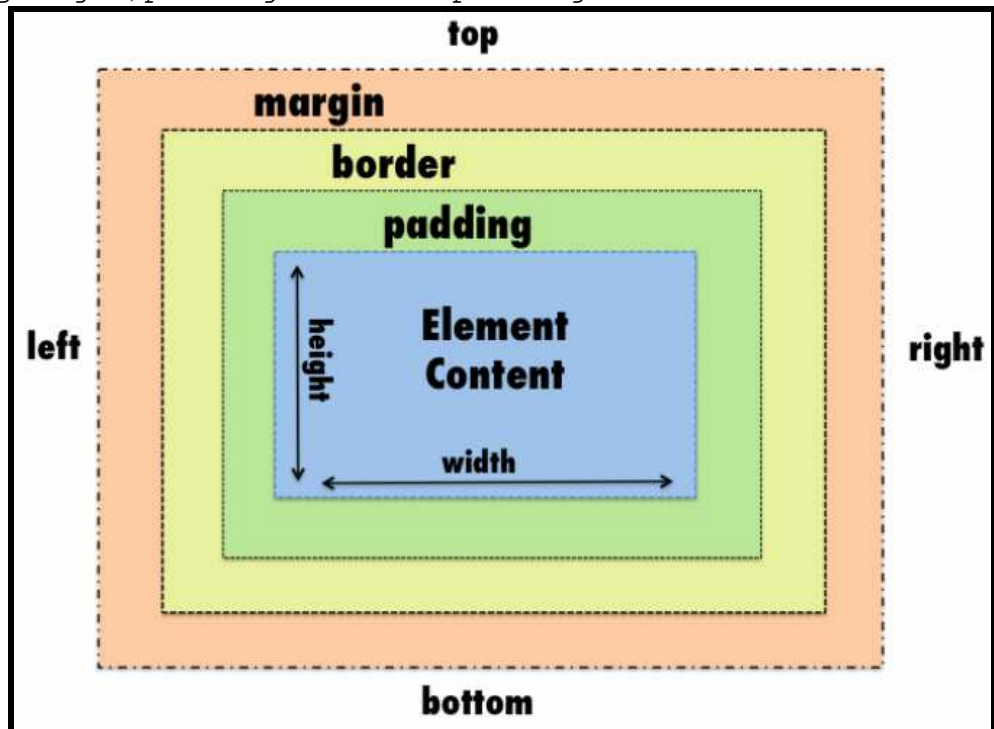
3.2.5. LES UNITES DE MESURE EN CSS

CSS offre différentes unités pour exprimer les dimensions. Certaines proviennent de la typographie, comme le point (pt) et le pica (pc), d'autres sont connues pour leur usage quotidien, comme le centimètre (cm) et le pouce (in). Et il y a également celle inventée spécifiquement pour CSS: le pixel (px). Les unités n'ont rien à voir avec les propriétés, mais avec le média de sortie: écran ou papier. Il n'y a pas de restriction à utiliser telle unité à tel ou tel endroit. Si une propriété accepte une valeur en px (margin: 5px) elle accepte également une valeur en pouces ou en centimètres (margin: 1.2in ou margin: 0.5cm) et vice-versa.

3.2.6. PADDING ET MARGIN

La propriété `margin` définit la taille des marges sur les quatre côtés de l'élément. Elle sont donnée dans le sens horaire en partant du haut. C'est une propriété raccourcie qui permet de manipuler les autres propriétés de marges : `margin-top`, `margin-right`, `margin-bottom` et `margin-left`. Il est possible d'utiliser des valeurs négatives pour chacun des côtés.

La propriété `padding` est une propriété raccourcie qui permet de définir les différents écarts de remplissage sur les quatre côtés d'un élément. Elle synthétise `padding-top`, `padding-right`, `padding-bottom` et `padding-left`.



Afin de visualiser les effets de `margin` et de `padding`, vous pouvez visiter les liens suivants :

<https://developer.mozilla.org/fr/docs/Web/CSS/padding>

<https://developer.mozilla.org/fr/docs/Web/CSS/margin>

Ecrivez les codes HTML et CSS permettant d'obtenir l'affichage suivant :

Fond (couleur purple)

En tête (2000px, couleur #FF9900, padding 50 px partout, margin 50px en haut et 0px ailleurs, centré)

Partie de gauche (750px, couleur #FF3366, padding 10px partout, margin 50px en haut et 0px ailleurs, hauteur min 400px, aligné à gauche)

Partie de droite (margin 50px en haut, 850px à gauche et 0px ailleurs, couleur #9966FF, padding 10px partout, hauteur min 400px, aligné à droite)

Partie principale (2000px, hauteur min 500px, couleur red, padding 50 px partout, margin 50px en haut et 0px ailleurs, centré)

Pied de Page (2000px, couleur #669933, padding 50 px partout, centré)

3.3. JAVASCRIPT

L'objectif de cette partie n'est pas de faire de vous des codeurs de JavaScript experts, mais simplement de vous donner les clés qui vous permettront d'explorer ce langage.

JavaScript ou js est un langage de programmation coté client (tout se passe sur votre machine) à contrario de PHP qui lui est déporté coté serveur.

C'est un langage interprété qui va nous permettre de créer de l'interaction entre le client et le serveur (dialogue client-serveur). Il permet de rendre une page dynamique.

3.3.1. LA PLACE DU JAVASCRIPT

Le JavaScript se place à trois endroits différents mais le placer en fin de fichier accélère le rendu de la page :

- Dans l'élément head d'une page HTML,
- Dans l'élément body d'une page HTML,
- Dans un fichier portant l'extension .js séparé.

3.3.1.1. DANS L'ELEMENT HEAD

Il faudra placer le JavaScript à l'intérieur d'une balise script (dans un élément script). On le placera généralement dans l'élément head d'un code HTML.

```
<script>
    alert("Message pour l'utilisateur !!! script dans le
                                         head) ");
</script>
```

Que constatez vous en exécutant le code HTML contenant le code précédent ?

3.3.1.2. DANS L'ELEMENT BODY

Il faudra placer le JavaScript à l'intérieur d'une balise script (dans un élément script) d'un code HTML..

```
<script>
    alert("Message pour l'utilisateur !!! script dans le
                                         body) ");
</script>
```

Que constatez vous en exécutant le code HTML contenant le code précédent ?

Que constatez vous en déplaçant cette partie de code dans l'élément body du code HTML ?

3.3.1.3. DANS UN FICHIER .JS SEPARÉ

C'est la méthode recommandée car elle permet la séparation des langages et la réutilisation d'un même code JavaScript dans plusieurs fichiers HTML.

Dans ce cas, nous allons devoir lier nos fichiers HTML et JavaScript en utilisant à nouveau un élément script et son attribut src.

Placez ceci dans le body du code HTML, « TP16Exo9_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo9_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo9_html_liier_js.js » :

```
alert("Message pour l'utilisateur !!! (script dans un fichier
                                         Javascript) ")
```

Que constatez vous en exécutant le code HTML contenant le code précédent ?

3.3.2. LES BLOCS, LES IDENTATIONS ET LES COMMENTAIRES

Le JavaScript doit être indenté et commenté (`//` sur une ligne ou `/*` et `*/` sur plusieurs lignes). Un bloc en JavaScript est délimité par des accolades `{ }`.

3.3.3. LA CONSOLE

Nous allons travailler avec un fichier HTML de votre choix. Les navigateurs sont dotés de consoles de développement permettant d'entrer des instructions à la volée, de consulter les données en mémoire ou d'explorer les fonctions et variables disponibles. On accède à la console par un menu pour les développeurs, à l'aide des touches de raccourci :

Navigateurs	Raccourcis
Exploreurs	F12 ou « outils » puis « outils de développement »
Firefox	« Menu » puis « développement web » puis « outils de développement »
Chrome	« Menu » puis « plus d'outils » puis « outils de développement »
Edge	F12 ou « outils supplémentaires » puis « outils de développement »
Opéra	Ctrl+Shift+I ou via le « menu » puis « outils de développement » puis « opera dragonfly »
Safari	Activer dans les options avancées, en cochant la case « Afficher le menu développement dans la barre des menus » puis Ctrl+Alt+I ou Ctrl+Alt+C

Placez ceci dans le body du code HTML, « TP16Exo10_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo10_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo10_html_liier_js.js » :

```
alert("Tapez F12 pour voir le message écrit dans la console
      (script dans un fichier JavaScript)");
console.log("Voici le message que j'écris dans la console
            !!!");
```

Que constatez vous en exécutant le code HTML contenant le code précédent ?

Que constatez vous en ouvrant la console de votre navigateur ?

Dans la console, saisissez les instructions suivantes et précisez ce que vous constatez en les exécutant :

```
console.log("Bonjour");
```

Que constatez vous ?

Dans la console, saisissez les instructions suivantes et précisez ce que vous constatez en les exécutant :

```
var fruit = "kiwis";
var mavariable = "J'aime les "
console.log(mavariable+fruit);
```

Que constatez vous ?

Dans la console, saisissez les instructions suivantes et précisez ce que vous constatez en les exécutant :

```
var fruits = [
    { name: "Kiwi", vitamine: "C" },
    { name: "Mangue", vitamine: "A" },
    { name: "Cassis", vitamine: "E" }
];
console.table(fruits);
```

Que constatez vous ?

Sur la page Web d'un site de votre choix et à l'aide de la console de votre navigateur, saisissez l'instruction suivante et précisez ce que vous constatez en l'exécutant :

```
document.querySelector('html').style.backgroundColor='purple';
```

Que constatez vous ?

Sur la page Web d'un site de votre choix et à l'aide de la console de votre navigateur, saisissez l'instruction suivante et précisez ce que vous constatez en l'exécutant :

```
var monImage = document.createElement('img');
monImage.setAttribute('src', 'https://c1.staticflickr.com/1/57
2/20463320350_58483f6bed.jpg');
document.querySelector('h1').appendChild(monImage);
```

Que constatez vous ?

Allez sur la page Web du lycée Turgot, de Google ou d'un site de votre choix et à l'aide de la console de votre navigateur, modifiez le code HTML pour modifier la couleur du texte ou du fond de l'article. Que constatez vous ?

A l'aide de la console de votre navigateur, explorez la constitution de cette page Web.

3.3.4. LES VARIABLES (ET LES CONSTANTES)

Une variable JavaScript est un conteneur servant à stocker temporairement une information. Il faut déclarer les variables. Leurs noms sont sensibles à la casse (majuscule / minuscule).

Il y a trois manières de déclarer une variable en JavaScript : `let`, `const` et `var`.

3.3.4.1.AVEC L'INSTRUCTION « LET »

Elle permet de déclarer une variable globale ou locale. On peut la déclarer sans valeur initiale et on peut la modifier après déclaration.

Placez ceci dans le body du code HTML, « TP16Exo11_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo11_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo11_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
JavaScript)");

let maVar = false;
console.log(maVar);
console.log(typeof(maVar));
let maVar1 = 2;
console.log(maVar1);
console.log(typeof(maVar1));
let maVar2 = 2.3;
console.log(maVar2);
console.log(typeof(maVar2));
let maVar3 = "a";
console.log(maVar3);
console.log(typeof(maVar3));
let maVar4;
console.log(maVar4);
console.log(typeof(maVar4));
maVar4=maVar+maVar3+maVar2;
console.log(maVar4);
console.log(typeof(maVar4));
maVar4=maVar1+maVar2;
console.log(maVar4);
console.log(typeof(maVar4));
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console.

C'est une variable globale si elle est déclarée en début de script :

Placez ceci dans le body du code HTML, « TP16Exo12_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo12_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo12_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
JavaScript)");
let maVar=1;
console.log(maVar);
if (true)
{
    maVar+=2;
    console.log(maVar);
}
maVar+=5;
console.log(maVar);
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console.

C'est une variable locale si elle est déclarée dans un bloc (fonction, for, if, ...) entre accolades {} :

Placez ceci dans le body du code HTML, « TP16Exo13_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo13_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo13_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
                                           JavaScript)");

if (true)
{
    let maVar=2;
    console.log(maVar);
}
console.log("En dehors du if");
maVar+=1;
console.log(maVar);
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console.

Placez ceci dans le body du code HTML, « TP16Exo14_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo14_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo14_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
                                           JavaScript)");

let maVar2 = 2;
console.log("maVar2 vaut avant le bloc :");
console.log(maVar2);
{
    maVar2=maVar2*4;
    console.log("maVar2 vaut dans le bloc :");
    console.log(maVar2);
    let maVar3 = 3;
    console.log("maVar3 vaut dans le bloc :");
    console.log(maVar3);
}
console.log("maVar2 à l'extérieur du bloc :");
console.log(maVar2);
console.log("maVar3 à l'extérieur du bloc :");
console.log(maVar3);
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console.

Modifiez le code pour qu'il affiche les 2 variables sans enlever les accolades.

Placez ceci dans le body du code HTML, « TP16Exo14_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo14bis_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo14bis_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
                                     JavaScript)");
for (let i = 0; i < 5; i++)    {
                                console.log("i dans la boucle
                                     for vaut : ");
                                console.log(i);
                                }
console.log("i en dehors de la boucle for vaut : ");
console.log(i);
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console. Modifiez le code pour qu'il n'affiche pas d'erreur sans enlever les accolades..

3.3.4.2.AVEC L'INSTRUCTION « CONST »

Elle permet de déclarer une constante globale ou locale. Elle doit être déclarée avec une valeur initiale et on ne peut pas la modifier après déclaration lorsqu'elle est un entier, un flottant ou une chaîne de caractère.

Placez ceci dans le body du code HTML, « TP16Exo16_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo16_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo16_html_liier_js.js » :

```
alert("Tapez F12 pour voir la console (script dans un fichier
                                     JavaScript)");

const maVar = false;
console.log(maVar);
console.log(typeof(maVar));
const maVar1 = 2;
console.log(maVar1);
console.log(typeof(maVar1));
const maVar2 = 2.3;
console.log(maVar2);
console.log(typeof(maVar2));
const maVar3 = "a";
console.log(maVar3);
console.log(typeof(maVar3));
const maVar4 = 6;
console.log(maVar4);
console.log(typeof(maVar4));
maVar4=maVar+maVar3+maVar2 ;
console.log(maVar4);
console.log(typeof(maVar4));
maVar4=maVar1+maVar2;
```

```
console.log (maVar4) ;
console.log (typeof (maVar4)) ;
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console. Modifiez le code pour qu'il affiche toutes les constantes.

On peut la modifier une variable créée avec l'instruction « const » après déclaration lorsqu'elle est un tableau.

Placez ceci dans le body du code HTML, « TP16Exo18_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo18_html_liier_js.js">
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo18_html_liier_js.js » :

```
alert ("Tapez F12 pour voir la console (script dans un fichier
                                           JavaScript)");

const tabl = [];
tabl.push (1);
tabl.push (2);
tabl.push ("Le tableau augmente");
console.log (tabl);;
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Expliquez ce qui est inscrit dans la console. Que fait la méthode « push » ?

3.3.4.3. AVEC L'INSTRUCTION « VAR »

L'instruction var est à éviter depuis la version ES6 = ES2015.

« ES » est l'abréviation d'ECMAScript, le standard sur lequel repose JavaScript.

La mise à jour du standard JavaScript est désormais annuelle. Afin de mieux s'y repérer, vous verrez donc également une appellation basée sur l'année :

ES6 = ES2015

ES7 = ES2016

ES8 = ES2017

ES9 = ES2018

ES10 = ES2019

ES11 = ES2020

ES12 = ES2021 ...

3.3.4.4. EXERCICE

Ecrire un fichier HTML avec un fichier JavaScript séparé permettant d'afficher la valeur de $r = x^2 + 2yz$. Il faudra déclarer quatre variables x , y , z et r et afficher le résultat r à l'aide de la fonction `alert`. Pour demander les valeurs à l'utilisateur, vous devrez utiliser les fonctions `parseInt` ou `parseFloat` et `prompt` (voir encadré ci-dessous). Les variables x et z sont entières et les variables y et r sont décimales.

```
x = parseInt (prompt ("x", "Saisir la valeur entière de x ici"));
```

3.3.5. LES FONCTIONS

Une fonction est introduite par le mot clé `function` suivi des arguments, puis du code de la fonction entre accolades `{ }`. Le résultat de la fonction est indiqué par le mot-clé `return`. La déclaration peut se faire à l'intérieur de l'élément `head` dans un élément `script` ou bien directement dans un fichier JavaScript séparé. Mais la déclaration ne doit pas se faire à l'intérieur de l'élément `body`.

Cette procédure permet de déclarer une fonction, il faut ensuite afficher le résultat :

```
document.write("Message à afficher", nom de la variable à  
afficher);
```

Ecrivez le code HTML et JavaScript permettant d'obtenir l'affichage de la page ci-dessous. Vous devrez pour y parvenir :

- Ecrire une fonction en JavaScript qui à une valeur x renvoie x^2 .
 - Réaliser un tableau de valeur de la fonction précédente de 0 à 4 avec un pas de 1. (utilisez la balise `table`)
 - Vous pouvez incorporer du code JavaScript dans du code html.
 - Vous pouvez incorporer du code html dans votre code JavaScript.
- Tester le code suivant :

```
document.write("<h1>Ceci est un titre de niveau 1 en HTML  
dans le script du Javascript</h1>")
```

- Vous pouvez faire une boucle `for` en JavaScript (exemple page 22).

Tableau de valeur de la fonction carrée					
x	0	1	2	3	4
x ²	0	1	4	9	16

Dans le body, le tableau est :
0,0,1,1,2,4,3,9,4,16

3.3.6. LA METHODE PROMPT()

La méthode `prompt()` fournit un moyen simple de récupérer une information provenant de l'utilisateur, on parle alors de boîte de dialogue invites. La méthode `prompt()` requiert deux arguments :

- le texte d'invite
- la chaîne de caractères par défaut dans le champ de saisie

Placez ceci dans le `body` du code HTML, « TP16Exo21_html_liier_js.js » étant le nom du fichier JavaScript :

```
<script src="TP16Exo21_html_liier_js.js">  
</script>
```

Et placez ceci dans le fichier JavaScript « TP16Exo21_html_liier_js.js » :

```
function demande() {  
    x=prompt("Quel âge as tu?", "Saisie ici  
ton âge");  
    return x;  
}  
document.write("Tu as ");  
document.write(demande());  
document.write(" ans, tu es très très vieux");
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ?

Réalisez une fonction JavaScript, qui lise un nombre au clavier en utilisant la méthode `prompt()`. Qui calcule le carré de la valeur saisie en utilisant la fonction carrée vu ci-dessus puis qui affiche le carré du nombre saisi ainsi : « Le carré du nombre 9 est 81 ».

3.3.7. LES EVENEMENTS INTERACTION HOMME / MACHINE

Les événements sont des actions de l'utilisateur, qui vont pouvoir donner lieu à une interactivité. L'événement par excellence est le clic de souris, car c'est le seul que le HTML gère. Grâce au JavaScript il est possible d'associer des fonctions, des méthodes à des événements tels que le passage de la souris au-dessus d'une zone, le changement d'une valeur, ...

La syntaxe d'un gestionnaire d'événement est la suivante :

```
onEvenement="Action_Javascript_ou_Fonction();" 
```

Pour plus d'informations vous pouvez consulter les sites suivants :

https://www.w3schools.com/jsref/event_onclick.asp

https://www.w3schools.com/jsref/event_ondragenter.asp

Placez ceci dans le body du code HTML :

```
<p id="prem" onClick="laFonction()">Cliquez ici (sélecteur  
                                d'identification : prem)</p>  
<p id="deux" onClick="laFonction2()">Ou cliquez là (selecteur  
                                d'identification : deux)</p>  
<p onClick="alert('Vous avez cliqué sur le troisième !!!');">  
    Ou cliquez ici (ouvrir une fenêtre "alerte")</p>  
<p id="traîne" onDragLeave="laFonction3()">Sélectionnez et  
    déplacez (sélecteur d'identification : traîne)</p>  
<p id="passage" onMouseOver="laFonction4()">Passez la souris  
    (sélecteur d'identification : passage)</p>  
<script>  
    function laFonction() {  
        document.getElementById("prem").innerHTML = "Vous  
        avez cliqué sur le premier !!! (sélecteur  
        d'identification : prem)";  
    }  
    function laFonction2() {  
        document.getElementById("deux").innerHTML = "Vous  
        avez cliqué sur le deuxième (sélecteur  
        d'identification : deux)!!!";  
    }  
    function laFonction3() {  
        document.getElementById("traîne").innerHTML =  
        "Vous avez sélectionné et déplacé !!! (sélecteur  
        d'identification : traîne)";  
    }  
</script>
```

```
function laFonction4() {
    document.getElementById("passage").innerHTML =
        "Vous avez passé la souris !!! (sélecteur
        d'identification : passage)";
}
</script>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Testez d'autres évènements.

3.3.7.1. LA METHODE QUERYSELECTOR()

La méthode `querySelector()` sélectionne le premier élément dans le document correspondant au sélecteur (ou groupe de sélecteurs) spécifié, ou `null` si aucune correspondance n'est trouvée.

Pour plus d'informations vous pouvez consulter les sites suivants :

https://www.w3schools.com/jsref/met_document_queryselector.asp

<https://developer.mozilla.org/fr/docs/Web/API/Document/querySelector>

https://www.w3schools.com/jsref/met_document_queryselectorall.asp

Placez ceci dans le body du code HTML :

```
<p onClick="laFonction()">
    Cliquez ici (sans sélecteur, donc s'applique au premier
                                élément)
</p>
<p class="classe" onClick="laFonction2()">
    Ou cliquez là (sélecteur de classe "classe")
</p>
<p id="ident" onClick="laFonction3()">
    Ou cliquez ici (sélecteur d'identifiant ' "ident")
</p>
<div>
    <p>
        Première division
    </p>
</div>
<div>
    <p>
        Deuxième division
    </p>
</div>
<p onClick="laFonction4()">
    Ou cliquez encore ici (sans sélecteur, donc s'applique à
    la première division)
</p>
<p>
    'C'est l'histoire d'un garçon troublé ...
</p>
<p style="color: white" class="suite">
    ... car il s'avère que son ours en peluche a disparu ...
<br>
```

```
<br>
... mais finalement il le retrouve ...
<br>
<br>
... et c'est tant mieux.' '
</p>
<p onClick="laFonction5()">
    Cliquez ici pour connaître la suite de l'histoire
    (sélecteur de classe "suite")
</p>
</body>
<script>
    function laFonction() {
        document.querySelector("p").style.color =
            "red";
    }
    function laFonction2() {
        document.querySelector("p.classe").
            style.backgroundColor = "yellow";
    }
    function laFonction3() {
        document.querySelector("#ident").
            style.color = "blue";
    }
    function laFonction4() {
        document.querySelector(("div > p")).style.
            backgroundColor = "green";
    }
    function laFonction5() {
        document.querySelector("p.suite").style.
            backgroundColor = "black";
    }
</script>
</body>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Testez d'autres exemples.

Vous pouvez remarquer que s'il y a plusieurs paragraphes seul le premier élément dans le sélecteur p est modifié car la méthode `querySelector()` ne s'applique sur le premier sélecteur de la page. Pour remédier à cela il faut donner un sélecteur d'identifiant id ou un sélecteur de classe class au paragraphe concerné.

3.3.7.2.LES BOUTONS

Nous allons intégrer à notre page des boutons.

Placez ceci dans le body du code HTML :

```
<h3>
    Les boutons
</h3>
<p>
```

```

    Cliquez sur les boutons ci dessous :
</p>
<button type="button" id="bout" onclick="laFonction()">
    Cliquez ici
</button>
<br>
<button onclick="quitte()">
    Fermer la fenêtre
</button>
<br>
<button onclick="laFonction2()">
    Ou cliquez ici
</button>
<script>
    function laFonction() {
        let x = document.getElementById("bout");
        x.disabled = true;
    }
    function laFonction2() {
        let x = document.createElement("button");
        let t = document.createTextNode("Cliquez
                                          moi");
        x.appendChild(t);
        document.body.appendChild(x);
    }
    function quitte() {
        if (confirm("Veux tu vraiment fermer
                    cette fenêtre ?"))
        {
            alert("A bientôt alors !");
            close();
        }
    }
</script>

```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Testez d'autres exemples.

3.3.7.3. LES FORMULAIRES

Un formulaire HTML est entre les balises `<form>` et `</form>` et contient notamment des champs de saisie appelés `input` de différents types ou `select`.

Les attributs `value` seront des variables utilisées en JavaScript coté client ou côté serveur (PHP).

3.3.7.3.1. INTERACTION AVEC LES CHAMPS DE SAISIE INPUT

Il existe différents types de champs de saisie `input` : `tel`, `search`, `url`, `email`, `number`, `range`, `color`, `date`, `time`, `text`, ...

Pour plus d'informations vous pouvez consulter le site suivant :

<https://developer.mozilla.org/fr/docs/Web/HTML/Element/Input>

Placez ceci dans le body du code HTML :

```

<form>
    <p>Login : <input type="text" name="lgn"/></p>

```

```

<p>Mot de passe : <input type="password" name="mdp"/></p>
<p>Catégorie :
    Femme <input type="radio" name="sexe" value="fem"/>
    Homme <input type="radio" name="sexe" value="hom"/>
</p>
<p>Spécialité(s) préférée(s) : </br>
    NSI <input type="checkbox" name="inf" value="info"/>
    Maths <input type="checkbox" name="mat"
                                value="math"/>
    Autres <input type="checkbox" name="aut"
                                value="autr"/>
</p>
<p><input type="button" value="Cliquez ici" id="bout"
                                onclick="laFonction()"></p>
<p><input type="submit" value="Envoyer"></p>
</form>
<script>
    function laFonction() {
        let x = document.getElementById("bout");
        x.disabled = true;
    }
</script>

```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Testez d'autres types de champs de saisie input.

Remarques :

- Pour que les boutons soient mutuellement exclusifs, ils doivent avoir le même attribut name : dans l'exemple précédent : "sexe".
- le terme boutons « radio » vient des anciennes radio. On pouvait mémoriser les stations de radio et les sélectionner à l'aide de gros boutons munis de ressorts. Lorsqu'on enfonçait un bouton, un mécanisme ressortait le bouton qui avait été enfoncé auparavant : les boutons étaient ainsi mutuellement exclusifs.



source : <https://st2.depositphotos.com>

- Plutôt que d'utiliser une balise input de type bouton :

```

<p><input type="button" value="Cliquez ici" id="bout"
                                onclick="laFonction()">
</p>

```

On peut utiliser balise button (vu au paragraphe précédent) qui permet d'obtenir le même rendu visuel :

```

</p>
<button type="button" id="bout" onclick="laFonction()">
    Cliquez ici
</button>

```

3.3.7.3.2. INTERACTION AVEC LES CHAMPS SELECT

La liste déroulante select fait parti des éléments interactifs importants.

Pour plus d'informations vous pouvez consulter le site suivant :

https://www.w3schools.com/jsref/dom_obj_select.asp

Placez ceci dans le body du code HTML :

```
<h1>
  Sélectionnez votre spécialité
</h1>
<form>
  <select name="Spécialités">
    <option value="nsi">Numérique et sciences
informatiques</option>
    <option value="maths">Mathématiques</option>
    <option value="autres">Autres spécialités</option>
  </select>
</form>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Testez d'autres méthodes et d'autres propriétés de la liste déroulante select.

La liste déroulante select permet un seul choix par défaut mais on peut autoriser un choix multiple. Dans ce cas les valeurs sélectionnées sont traitées comme un tableau.

Placez ceci dans le body du code HTML :

```
<h1>
  Sélectionnez vos spécialités
</h1>
<h3>
  Pour faire des choix multiples, sélectionnez vos choix en
appuyant sur "ctrl"
</h3>
<form>
  <select name="Spécialités[]" multiple>
    <option value="nsi">Numérique et sciences
informatiques</option>
    <option value="maths">Mathématiques</option>
    <option value="autres">Autres spécialités</option>
  </select>
</form>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ?

Il est possible de sélectionner par défaut un (ou plusieurs éléments) en ajoutant l'attribut selected à la balise <option>.

Placez ceci dans le body du code HTML :

```
<h1>
  Sélectionnez vos spécialités
```

```

</h1>
<h3>
    Pour faire des choix multiples, sélectionnez vos choix en
    appuyant sur "ctrl"
</h3>
<form>
    <select name="Spécialités[]" multiple>
        <option value="nsi" selected> Numérique et sciences
            informatiques</option>
        <option value="maths"> Mathématiques</option>
        <option value="autres" selected> Autres
            spécialités</option>
    </select>
</form>

```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ?

3.3.7.3.3. EXERCICE

Ecrivez le code HTML et JavaScript permettant d'obtenir l'affichage de la page ci-dessous. Vous devrez pour y parvenir :

- utiliser les balises input et select,
- préciser le type lors de l'utilisation d'une balise input :

Affichage	Type de l'input
Nom	text
e-mail	email
Mot de passe	passwd
Numéro	text
Validité	date
Bouton « Valider le paiement »	submit

- préciser le name dans chacun des cas (attribut name utilisé plus tard pour retrouver la valeur d'un élément d'un formulaire).
- proposer trois modèles de carte bancaire : « Carte Visa », « Carte Bleue » (sélectionnée par défaut) et « Carte Mastercard ».
- le bouton « Valider le paiement » active une fonction du script Javascript. Cette fonction doit demander au client confirmation au message : « Voulez vous vraiment valider les renseignements ? ». Si le client confirme le message précédant alors le message suivant s'affiche « C'est validé !!! » sinon l'affichage est « Ce n'est pas validé ».

Formulaire de paiement
 Les champs obligatoires sont précédés par "*"

Informations de contact
 Qualité : Monsieur ☐ Madame ☐
 Nom : *
 e-mail : *
 Mot de passe : *

Informations de paiement
 Type de carte :
 Numéro : *
 Validité : *

 format de la validité: jj/mm/aaaa

3.3.7.4. RECUPERATION DES VALEURS DES FORMULAIRES

Nous allons récupérer les valeurs d'un formulaire et en faire l'usage au niveau client.
Placez ceci dans le body du code HTML :

```
<form>
  <p>
    Choix d'une nationalité :
  </p>
  <h6>
    Pensez à visualiser la console en appuyant sur F12
  </h6>
  <select id="choix" name="pays" onchange="selection()">
    <option value="ras">Choisir sa nationalité</option>
    <option value="fr">Français</option>
    <option value="cn">Chinois</option>
    <option value="it">Italien</option>
  </select>
  <p>
    Vous avez choisi comme nationalité :
    <span id="nat">
    </span>
  </p>
</form>
<script>
  function selection() {
    const selecteur = document.getElementById('choix');
    const monChoix = selecteur[selecteur.selectedIndex];
    // visualisation dans la console
    console.log(monChoix.index+' '+monChoix.value +' '+
                                                         monChoix.text);
    document.getElementById('nat').innerHTML =
                                                         monChoix.text;
  }
</script>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Que se passe-t-il dans la console (F12) ?

Dans le script précédent, grâce à la gestion des événements faite par `onchange`, chaque changement d'option lance la fonction `selection()`.

`getElementById('id')` est une méthode qui récupère l'objet de la page identifié par le sélecteur d'identifiant `'id'`. Donc `document.getElementById('choix')` récupère l'objet de la page en cours identifié par `'choix'` :

index	Value	text
0	ras	Choisir sa nationalité
1	fr	Français
2	cn	Chinois
3	it	Italien

`selectedIndex` est une méthode qui renvoie la valeur de l'option choisie dans une liste déroulante. C'est-à-dire que cette méthode indique le rang (de 0 à ...) de l'élément de la liste qui a été sélectionnée par l'utilisateur (dans notre cas de 0 à 3).

`selecteur.selectedIndex` renvoie l'indice du choix fait par l'utilisateur dans la liste déroulante nommée « `selecteur` » (dans notre cas un nombre entre 0 et 3 en fonction du choix fait).

`innerHTML` est une méthode qui permet de récupérer tout le contenu HTML d'un élément d'une page html.

`document.getElementById('nat').innerHTML` intègre le contenu HTML à l'emplacement de la page identifié par le sélecteur d'identifiant nommé 'nat' (dans notre cas une des 4 textes en fonction du choix fait).

3.3.7.4.1. EXERCICE 1

Ecrire un formulaire qui demande votre âge et qui indique dans la même page si vous êtes majeur ou mineur.

3.3.7.4.2. EXERCICE 2

Placez ceci dans le body du code HTML :

```
<form>
  <fieldset>
    <legend>Veuillez sélectionner les spécialités que
              vous conserverez l'année prochaine :
    </legend>
    <div>
      <input type="checkbox" id="nsi" name="interest"
              value="nsi">
      <label for="nsi">NSI</label>
    </div>
    <div>
      <input type="checkbox" id="ma" name="interest"
              value="ma">
      <label for="ma">Maths</label>
    </div>
    <div>
      <input type="checkbox" id="si" name="interest"
              value="si">
      <label for="si">SI</label>
    </div>
  </fieldset>
</form>
```

Commentez le code. Que constatez vous en exécutant le code HTML contenant le code précédent ? Ajoutez une spécialité supplémentaire à cette liste à cocher.

En vous inspirant de l'exercice précédent, affichez la ou les réponses dans la page HTML. Si la réponse contient NSI, la page HTML doit afficher : « Vous conservez NSI, bravo ceci est un très bon choix ! ». Dans les autres cas la page HTML doit afficher : « Vous conservez nom de la matière ».

4. INTERACTION CLIENT - SERVEUR HTTP

La structure client - serveur désigne un mode de communication entre programmes :

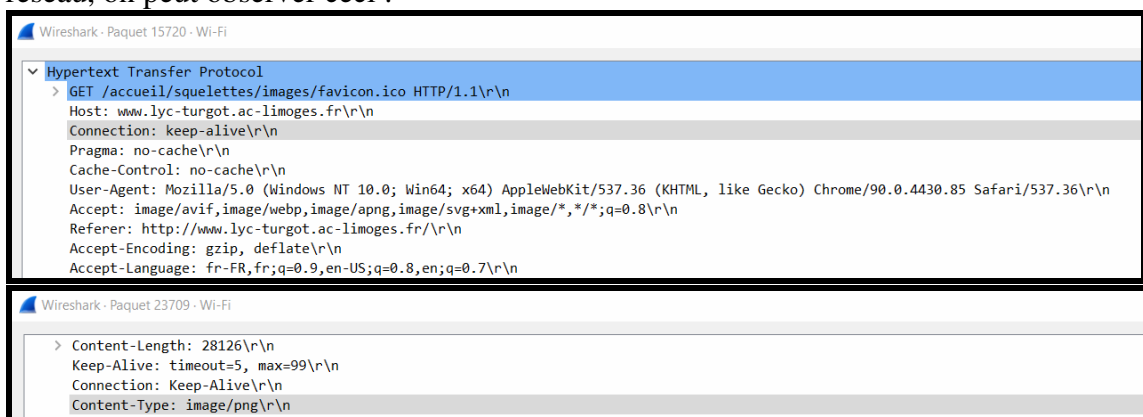
- l'un qualifié de client, envoie des requêtes,
- l'autre, le serveur, y répond.

4.1. PROTOCOLE HTTP

Dans le cas du Web, le client est le navigateur de l'ordinateur utilisé, le serveur est un ordinateur du réseau Internet (on ne connaît pas sa position géographique) et le protocole utilisé est HTTP.

Le protocole HTTP(Hyper Text Transfer Protocol) est un protocole du Web d'échange entre un client et un serveur Web

En utilisant Wireshark, logiciel permettant d'observer les échanges qui transitent sur un réseau, on peut observer ceci :



- User-Agent : donne le client et la version utilisé pour ce protocole, Chrome dans ce cas.
- GET : la méthode utilisée.
- Host : URL (Uniform Resource Locator) adresse Web, site du lycée dans ce cas.
- Content-type : le type de fichier renvoyé par le serveur, un .png dans ce cas.

Les différentes méthodes applicables au protocole HTTP sont GET, HEAD, POST, OPTIONS, CONNECT, TRACE, PUT, PATCH, DELETE

- GET : demande des informations et la ressource désignée. Les paramètres de la demande sont transmissent dans l'URL.
- POST : C'est une méthode qui permet de soumettre des données en vue d'un traitement (côté serveur). C'est cette méthode que nous utiliserons pour envoyer au serveur les données issues d'un formulaire.
- HEAD : demande des informations concernant la ressource

4.2. PROTOCOLE HTTP SECURISE

Le souci du protocole HTTP est qu'il est transparent (on peut observer et récupérer les informations qui circulent). Ceci pose des problèmes de sécurité évidents (échanges bancaires, vie privée...). La réponse à ce problème est le protocole HTTPS (S est pour Secure).

Le protocole HTTPS se compose de 2 protocoles:

- Le protocole HTTP
- Le protocole de sécurisation des échanges sur Internet SSL / TLS. Ce protocole va créer une sorte de canal sécurisé entre le client et le serveur. SSL (Secure Socket Layer) comme son nom l'indique est une couche supplémentaire sécurisée (actuellement obsolète car vulnérable aux

attaques Man In The Middle (MITM) rendant possible l'interception). Le protocole utilisé est le TLS (Transport Layer Security).

Grâce un échange de clés entre eux, le serveur et le client vont établir une connexion chiffrée dont eux seuls pourront lire le contenu. Car seul le client et le serveur en possession de la clé de décryptage pourront déchiffrer les données reçues. Pour plus d'informations consultez :

<https://www.youtube.com/watch?v=7W7WPMX7arI>

4.3. INTERACTION CLIENT - SERVEUR METHODES GET ET POST

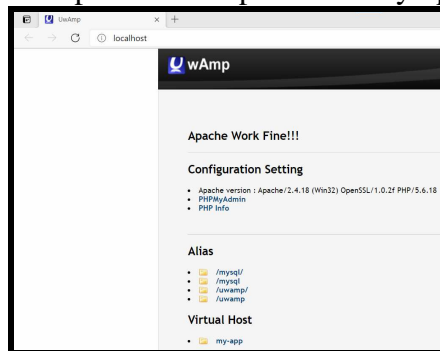
Afin de comprendre l'interaction client – serveur, nous allons utiliser le logiciel permettant de simuler un serveur : Uwamp. Tous les fichiers de travail devront se trouver dans le répertoire « my-app ».



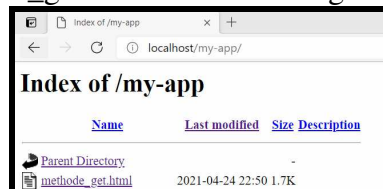
4.3.1. INTERACTION CLIENT - SERVEUR METHODE GET

Cliquez sur « Dossier www » puis mettez dans le répertoire « my-app » les fichiers « methode_get.html » et « reponse_methode_get.php ». Fermez l'explorateur de fichiers.

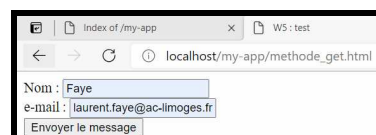
Cliquez sur « Navigateur www » puis sur le répertoire « my-app » de l'affichage suivant :



Cliquez sur le fichier « methode_get.html » de l'affichage suivant :



Complétez les champs suivants et validez en cliquant sur le bouton « Envoyer le message » :



Recopier les informations contenues dans le champ « URL » du navigateur :

Que constatez-vous dans l’affichage de la page Web et dans le champ « URL » du navigateur ?

Où se trouvent les informations transférées par ce formulaire ?

4.3.2. EXERCICE METHODE GET

Reprenez les fichiers précédents et modifier le formulaire pour faire un formulaire qui ressemble à l’affichage suivant (au démarrage NSI est cochée) :

Réalisez une réponse qui ressemble à l’affichage suivant si les 4 spécialités sont cochées :

Si aucune spécialité a été cochée, votre réponse ne doit afficher aucune ligne, si une spécialité est cochée une ligne est affichée, si deux spécialités sont cochées deux lignes sont affichées, ... Utilisez une boucle if en PHP et la fonction « empty ».

4.3.3. INTERACTION CLIENT - SERVEUR METHODE POST

Pour utiliser la méthode **POST** dans un formulaire, il suffit de remplacer l’argument **method="GET"** de la balise **form** par **method="POST"**. Il faut également remplacer le nom du fichier réponse dans l’argument **action="reponse_methode_get.php"** par **action="reponse_methode_post.php"**. Modifiez les balises **form** et **title** du fichier « methode_get.html » puis sauvegardez le sous « methode_post.html ». Modifiez les balises **title** et **body** (en remplaçant **\$_GET[user_name]** par **\$_POST[user_name]** et **\$_POST[user_mail]** par **\$_POST[user_mail]**) du fichier « reponse_methode_get.php » puis sauvegardez le sous « reponse_methode_post.php ».

Avec UwAmp, cliquez sur « Dossier www » puis mettez dans le répertoire « my-app » les fichiers « methode_post.html » et « reponse_methode_post.php ». Fermez l’explorateur de fichiers. Cliquez sur « Navigateur www » puis sur le répertoire « my-app ». Cliquez sur le fichier « methode_post.html ». Complétez les champs et validez en cliquant sur le bouton « Envoyer le message ».

Recopier les informations contenues dans le champ « URL » du navigateur :

Que constatez-vous dans l’affichage de la page Web et dans le champ « URL » du navigateur ?

Afin de visualiser les informations transférées par ce formulaire, nous allons accéder à l’outil réseau de votre navigateur (voir page 18 de ce cours) avec F12 ou « outils » puis « outils de développement ». Exécutez le fichier « methode_post.html » sous UwAmp, accédez à l’outil réseau de votre navigateur, actualisez la page Web avec . Dans l’outil réseau, sélectionnez « reponse_methode_post.php » puis « données du formulaire ».

Quelles sont les informations transférées par ce formulaire ?

Quelle différence observez vous entre les méthodes « GET » et « POST »?

La méthode « GET » est à choisir si on veut pouvoir partager une adresse (moteur de recherche par exemple). On peut choisir la méthode post quand on ne veut pas que URL soit modifiée.

4.3.4. EXERCICE METHODE POST

Modifiez les fichiers html et php l’exercice 4.3.2 pour qu’ils fonctionnent avec la méthode « POST ».

5. TRAVAIL A FAIRE

Votre travail est de réaliser un script HTML, CSS, JavaScript commenté que vous intégrerez dans votre site permettant de faire le bilan du travail fait cette année sous la forme d’un carnet de bord.

Vous pouvez réaliser des photos, des sons ou des vidéos de deux minutes maximums qui montrent votre travail (code et effet). Pour y parvenir, prenez appui sur ce cours et fouillez sur Internet. Vous pouvez y intégrer les idées suivantes :

- Ecrire des fonctions qui donnent :
 - le nombre de clics sur la page Web,
 - le temps passé sur la page Web,
 - Le nombre de visite sur la page Web, ...
- Cacher du texte et le faire apparaître avec un clic (pour une solution d'exercice),
- Cacher du texte dans du texte (pour alléger une page en supprimant les définitions des termes), ...